

Plugging the Team In

The AGENTM2M Plugin for Claude Code, its MCP Server and the Hosted Service,
Explained for Software Engineering Students

*Companion to “When the Team Writes Itself” (the AUTO M2M teaching edition) [1]
and to “AGENTM2M: M2M Transformations for Agentic Collaborations in Software Engineering” [2]*

Abstract. AGENTM2M runs a team of AI agents whose hand-offs are checked by a program: a deterministic engine decides what each agent must produce, and a large language model (LLM) only fills in individual values, each of which a validator must accept. AUTO M2M lets an LLM design the whole team and admits the design only if a checker approves it. This document explains how both were turned from a research library into tools a developer uses every day: a *plugin* for the coding assistant Claude Code, a server that speaks the *Model Context Protocol* (MCP), and a *hosted service* that offers the same tools over the web. Its central idea is *host mode*: Claude Code does all the LLM work, but only by answering prompts the engine builds, and the engine keeps every decision. We build all background from what a software-engineering undergraduate knows, follow two complete sessions whose outputs come from the real engine, and close with security, testing, and an honest account of what is not solved.

Contents

1	How to read this document	2
2	Background you need: a short course	3
3	The running example: a four-agent development team	8
4	The problem: from a research library to a tool	12
5	The Model Context Protocol (MCP)	13
6	Claude Code plugins	17
7	Architecture of the plugin	19
8	Host mode: Claude Code plays the agents, the engine decides	22
9	Skills, subagents and hooks	25
10	Walkthrough 1: a hand-written team in the plugin	28
11	Walkthrough 2: AUTO M2M in the plugin	33
12	The hosted service	39
13	Security and trust	43
14	Building, testing and releasing the plugin	45
15	Requirements revisited, and what is still open	46
16	Exercises	47
A	Reference: the 24 tools	48
B	Try it yourself	51

C	The admitted team of Walkthrough 2	52
D	Glossary	52
E	Answers to the questions	56
F	Hints for the exercises	57

1 How to read this document

Who this is for. An undergraduate who has taken a first software engineering course: you know classes and interfaces, unit tests, JSON, and roughly what a web API is. You do not need to know large language models, the Model Context Protocol, Claude Code, or model-driven engineering; Section 2 teaches what you need. The companion AutoM2M teaching edition [1] explains *why* agent teams need checked interfaces; this document explains *how* that idea reaches a developer’s editor. You can read it without the companion.

What you will be able to do. After reading this document you can:

1. explain, message by message, how an AI assistant discovers and calls the tools of an MCP server;
2. name the four extension points of a Claude Code plugin and say what the AGENTM2M plugin puts in each;
3. explain why *host mode* lets Claude Code do all the LLM work without weakening any guarantee of the engine, and where that argument has limits;
4. read a hand-off rule, predict which values a requirement change obliges to be redone, and check your prediction with *impact*;
5. explain how the hosted service keeps different users’ data and code apart, and why it refuses to run code when it cannot isolate it.

The route through the document. Section 2 is a short course on the background. Section 3 introduces the running example, a four-agent development team, and teaches you to read its files. Section 4 states the problem as seven requirements. Sections 5 and 6 explain the two technologies the solution is built from: MCP and Claude Code plugins. Sections 7 to 9 explain the plugin: its architecture, its central idea (*host mode*), and how skills, subagents and hooks enforce the rules. Sections 10 and 11 walk through two complete sessions. Sections 12 and 13 explain the hosted service and the security model, Section 14 how the plugin is built and tested, and Section 15 revisits the requirements and what is still open. Section 16 has graded exercises. The appendices hold a reference of all 24 tools, a “try it yourself” guide, the full team of the second walkthrough, a glossary, and answers.

Four reading paths. *In 30 minutes:* Section 4, Section 8 up to Section 8.2, Figures 4, 5, 8 and 9, and the green takeaway boxes. *Hands-on:* Section 3, Appendix B, then Section 10 while you run it. *Security:* Sections 2.9, 12 and 13. *In full:* in order, answering each orange question before checking Appendix E.

Boxes. Blue boxes are *remarks* that sharpen an idea. Orange boxes (“Check your understanding”) ask you a question; answer it before reading on. Green boxes summarise the *key takeaways*. “In plain words” lines restate a hard idea without jargon.

Colours. Listings and figures use one colour code:

Person	A file or text a person authors: a task, a command, a configuration file, a skill. In figures: you, and files you edit.
Code	Source code: Python, rule modules, typed teams. In figures: the plugin and server code.
LLM	A prompt for the LLM, or its answer. In figures: Claude Code and its subagents.
Engine	Deterministic output of the engine, checker or server. In figures: the engine and the files only it writes.
Wire	A protocol message between two programs (line breaks added).
Refused	In figures only: an escalated value or a refused request.

White boxes in figures are external programs (a browser, curl). In protocol listings, `->` marks a message from client to server and `<-` one from server to client.

What kind of claim each statement is. Every listing caption starts with its kind (Table 1).

Table 1. Kinds of claims, and how captions mark them.

Caption starts with	What it means	Where
<i>Observed</i>	Produced by the real engine, version 0.3.0, in host mode, by the scripts in docs/plugin-walkthroughs/, which call the same functions the MCP tools call. Outputs are <i>abridged</i> : we remove keys and shorten text (marked ...) but never change a value. Digests are the real ones. “Rendered” means we reformatted JSON as key: value lines.	Sections 5 and 9 to 12
<i>Code excerpt</i>	Copied from the repository (file named in the caption), sometimes shortened.	throughout
<i>Illustrative</i>	Made by us to explain: the wallet task, the values we typed in place of an LLM, the shape of a protocol message.	marked
(no listing)	<i>Design explanations</i> say why the code is the way it is. Where the reason is ours rather than the code’s or its documentation’s, we say so.	throughout

Remark 1 (Who typed the answers). In both walkthroughs the values an LLM would normally write (API signatures, tests, code) were typed by us, some of them deliberately wrong, so that you can see what the engine does with a wrong answer. Everything the *engine* printed in response is real, and you can reproduce it with the scripts in docs/plugin-walkthroughs/ (Appendix B).

2 Background you need: a short course

Skip any subsection you already know.

2.1 LLMs and agents in one page

A *large language model* (LLM) is a program that, given some text (the *prompt*), produces a likely continuation. Asking it once is called *drawing a sample*. Four properties matter here.

- **It is not deterministic.** Two samples for the same prompt can differ. A setting called *temperature* controls how much: at low temperature, the same prompt gives nearly the same answer.
- **It can be confidently wrong.** A sample can look right and be false, for example code that compiles but computes the wrong thing.
- **It only knows its context.** Everything the LLM can see in one call (instructions, conversation so far, tool results) is its *context*. Whatever is not in the context, or in its training, is invisible to it.

- **It can be steered by text only.** You cannot “forbid” an LLM to use something it has seen; you can only avoid showing it.

An *agent* is an LLM in a loop with *tools*: the program shows the LLM a goal and a list of tools (read a file, run a test); the LLM picks an action; the program performs it and adds the result to the context; and so on, until the LLM says it is finished. A *team* is several agents whose outputs become each other’s inputs; one agent passing work to the next is a *hand-off*.

2.2 A coding assistant is an agent in your terminal

Claude Code is an agent that works in a software project: it reads and edits files, runs commands and tests, and talks to you in a terminal or an editor [3]. Its built-in tools include Read, Edit, Write, Glob (find files by name), Grep (search file contents) and Bash (run a shell command). Before it uses a tool that changes something or that comes from an extension, Claude Code normally asks you for permission. Three more properties matter here.

- **It can be extended** without changing Claude Code itself, by installing a *plugin* (Section 6).
- **It can start helpers.** Claude Code can hand a sub-task to a *subagent*: a second agent with its own instructions, its own (often smaller) list of allowed tools, and a fresh context. The main agent sees only the subagent’s final report.
- **It sees a lot.** By default it can read the whole project and remembers the whole conversation. Usually a strength; for AGENTM2M a danger, because an agent that fills one value must see *only* that value’s inputs (Section 2.7). Section 9 shows how the plugin resolves this tension.

In the vocabulary of MCP (Section 5), Claude Code is the *host*: the program in which everything else runs and which owns the LLM.

2.3 Processes, standard input and output

When Claude Code starts a helper program, the operating system creates a new *process*. Every process has three standard streams: *standard input* (stdin), which it reads; *standard output* (stdout), which it writes; and *standard error* (stderr), for diagnostics. The parent can write into the child’s stdin and read its stdout, like a pipe in `cat file | grep x`. The AGENTM2M plugin uses exactly this: Claude Code starts the AGENTM2M server as a child process and exchanges JSON messages with it, one per line, over stdin and stdout.

2.4 JSON, JSON Schema and remote procedure calls

JSON is a text format for structured data: objects `{"key": value}`, arrays `[1, 2]`, strings, numbers, booleans and `null`. A *JSON Schema* is a JSON document that says which JSON documents are allowed, for example “an object with a required string field `target_key`”. It is to JSON what a class declaration is to an object.

A *remote procedure call* (RPC) lets one program call a function in another program as if it were local: the caller sends the function’s name and arguments; the callee runs it and sends back the result or an error. JSON-RPC 2.0 [4] is a minimal RPC protocol in JSON with three kinds of messages (Listing 1).

Listing 1. Illustrative: the three kinds of JSON-RPC 2.0 messages.

```
request    {"jsonrpc": "2.0", "id": 7, "method": "add", "params": {"a": 2, "b": 3}}
response   {"jsonrpc": "2.0", "id": 7, "result": 5}
or         {"jsonrpc": "2.0", "id": 7, "error": {"code": -32601, "message": "Method not found"}}
notification {"jsonrpc": "2.0", "method": "log", "params": {"text": "hello"}}    (no id: no answer)
```

The *id* pairs each response with its request, because several requests may be in flight at once. A *notification* has no *id* and gets no response.

Q1. Check your understanding. Why must the response carry the same *id* as its request? What could go wrong if two requests were outstanding and responses had no *id*?

2.5 HTTP, REST, API keys and hashes

Programs on *different* machines usually talk over *HTTP*: a client sends a request (a method such as GET or POST, a path such as `/api/auto/run`, *headers* and an optional body) and the server returns a status code (200 OK, 401 unauthorized, 403 forbidden, 404 not found, 409 conflict, 500 server error, ...), headers and a body [5]. A *REST API* [6] exposes resources as paths and operations as HTTP methods.

A *cryptographic hash* such as SHA-256 turns any data into a short, fixed-length string, its *digest*. Equal data always gives the equal digest; any change, however small, gives a different one; and you cannot recover the data from the digest. We use digests twice. First, a server that serves many users identifies them by *API keys*: long random secrets that the server gives each user once and that the user sends with every request, usually as the header `Authorization: Bearer <key>`. A careful server stores only the *digest* of each key: when a request arrives, it hashes the presented key and looks the digest up. If the database leaks, the keys cannot be recovered. Second, the engine uses digests to notice that the inputs of a value have changed (Section 2.7).

2.6 Plugins and inversion of control

A *plugin* adds behaviour to a host application through *extension points* that the host defines; you know this from IDE extensions. The host stays in charge: it decides when to load the plugin and when to call it (*inversion of control*: “don’t call us, we’ll call you”). A plugin can do nothing the extension points do not allow. Claude Code offers four extension points (tools through MCP servers, skills, subagents, hooks) and one distribution channel (marketplaces).

2.7 AGENTM2M in one example

This subsection and the next condense the companion documents [2, 1]. Figure 1 shows the whole idea on one value.

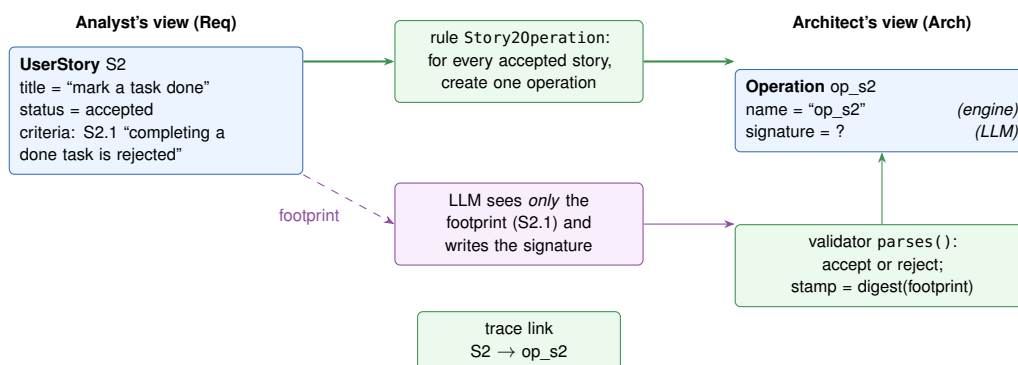


Figure 1. AGENTM2M on one value. The engine (green) decides that `op_s2` exists and fills its name; the LLM (purple) writes only the signature, from the footprint alone; a validator decides whether that answer is kept; the engine records a trace link and a stamp.

Views. Each agent of a team owns one *view*: a model that conforms to a small class diagram, its *metamodel*. Think of the metamodel as a *form* and the view as the *filled-in form*. Only the owner may change its view: these are its *write rights* (written ω in the AGENTM2M paper, and ω in error messages).

Hand-offs are rules. A hand-off from one view to another is a *model-to-model (M2M) transformation*: a program that reads one model and produces another. It is written as rules in the style of the ATL language [7]: “for every accepted user story, create one API operation”. A deterministic *engine* runs the rules: it decides which target objects exist and how they link. A field computed by a formula is a *structural binding*. A field no formula can compute (an API signature, a test) is a *stochastic binding*, written `@llm(prompt, footprint)`: an LLM writes it. In this document the *binding* is the clause in the rule; the *value* is what gets written into one target object.

Footprint and validator. The *footprint* is the exact data the LLM may see when it writes one value. The value is accepted only if its *validator* (the `@check` clause) passes. A *form* validator checks shape (“does it parse?”); a *behaviour* validator executes the value (“do the tests pass?”). After k rejections on the same footprint (here $k = 3$) the value is *escalated*: the engine stops asking and reports it, instead of looping forever.

Trace links, stamps and obligations. Every time a rule fires, the engine records a *trace link* (“this target came from that source, through this rule”). With every accepted value it stores a *stamp*: the digest of the footprint the value was written from. When a source changes, the values whose stamp no longer equals the digest of their current footprint are *stale*; redoing them are the *obligations* of the change. Values whose footprint did not change are left alone.

Fixpoint and φ . The engine runs all hand-offs again and again until a pass creates and changes nothing: that state is a *fixpoint*. The team is done when the *acceptance predicate* φ holds: every rule match has its target, every value is accepted for its current footprint, and nothing is pending or escalated. An agent cannot end the run by claiming success.

2.8 AUTOM2M in one example

In AGENTM2M a person writes views and rules. In AUTOM2M a *builder* LLM writes the whole team design as one JSON document, a *typed team*. A deterministic *checker* admits it only if six conditions hold (Table 2); otherwise it returns exact *diagnostics* and the builder revises.

The task is a Python class with unimplemented methods; it is turned (*lifted*) into a fixed Goal view with one Method object per method to implement. For an AUTOM2M team, φ is the conjunction of four clauses:

- `cover(G)`: every goal object in G (here every method) has a validated descendant, and every method has an accepted deliverable (code);
- `valid`: every accepted value still passes its validators on the final models;
- `fresh`: every stamp equals the digest of its current footprint;
- `no0bl`: no value is pending or escalated.

If φ fails, the engine *locates* the failing value by trace lookup, and a repair, a revised team called a *delta*, must pass the same checker before it is applied.

Remark 2 (The one sentence to remember). In AGENTM2M an LLM writes values; the engine owns structure. In AUTOM2M an LLM proposes the team; the checker owns admission. In the plugin Claude Code does all the LLM work; the engine keeps every decision.

Words we use. A **binding** is a clause in a rule; a **value** is what it writes into one object. A value is **pending** when it is ready to be written (never accepted, or **stale**); **blocked** when its footprint still contains an upstream value that was never accepted; an **obligation** when it is pending because something changed. **Engine LLM** means an LLM the engine calls itself through an API; in **host mode** there is none, and Claude Code writes the values. An **agent** is a member of the team (Analyst, Architect, ...); Claude Code plays the agents.

Table 2. The six admission conditions, with an example violation each (all but W3 are real diagnostics from this document).

	Meaning	Example violation
W1	Well typed: rules read and write only fields that exist; every LLM value has a known validator.	footprint path 'd.method.returnType': Goal!Method has no feature 'returnType'
W2	One writer: each view has one owning agent; each class is created by one rule.	view Test written by ['Developer', 'Tester']
W3	Complete: every mandatory field is filled exactly once.	a rule that never fills the mandatory name
W4	Anchored coverage: every goal flows into a value checked by a <i>behaviour</i> validator; the deliverable is behaviour-checked; every view lies on such a flow.	view Code reaches no behavioural check
W5	The engine decides “done”: φ is exactly the four engine clauses; no cycles between views.	done-clause Tester.says('ALL TESTS PASS') is not engine-checkable
W6	Right tools: the owner of a value has the tools its validators need.	Method2Test.code needs ['exec']; owner Tester has []

2.9 Isolation and sandboxes

Behaviour validators *execute* code that an LLM wrote. Executing untrusted code is dangerous: it could delete files, read secrets, or use the network. A *sandbox* runs code with restricted powers. A plain *child process* with resource limits (CPU time, memory, file size) stops runaway code but can still read every file the user can. Linux *namespaces* let a process get its own private view of the network, the process list or the file system; a namespace sandbox such as bubblewrap [8] uses them to give code no network and only a few read-only directories, so other users’ files simply are not there. The principle behind both is *least privilege* [9]: give each part of a system only the rights it needs.

Key takeaways.

- An LLM produces samples from its context; an agent is an LLM with tools; Claude Code is an agent and the *host* in which the plugin runs.
- Programs talk through pipes (same machine) or HTTP (different machines); JSON-RPC pairs requests and responses by id.
- A digest changes whenever the data changes: servers store digests of API keys, and the engine stores digests of footprints (stamps).
- AGENTM2M: the engine builds structure, the LLM fills footprint-bounded values, validators decide, stamps make change impact exact, φ decides “done”. AUTOM2M: a checker admits LLM-proposed teams (W1–W6).

3 The running example: a four-agent development team

The plugin ships three ready-made teams, called *templates*. We use the first, DEVTEAM, the running example of the AGENTM2M paper, throughout. This section introduces it and teaches you to read its two kinds of files: the team description `team.yaml` and the rule modules.

3.1 The team

DEVTEAM builds a small to-do web service (Figure 2). The *Analyst* owns the view **Req**: user stories with acceptance criteria. The *Architect* owns **Arch**: API operations. The *Developer* owns **Code**: code edits. The *Tester* owns **Test**: test cases. The starting (*seed*) model has three stories: S1 *create a task* (criterion S1.1 “a title is required”), S2 *mark a task done* (S2.1 “completing a done task is rejected”), both accepted, and S3 *export tasks as CSV* (S3.1 “the file has a header row”), still a draft.

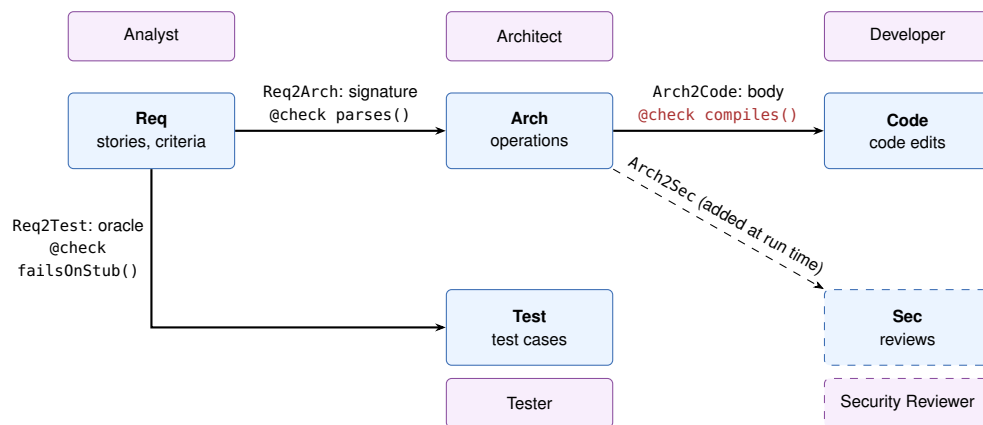


Figure 2. DEVTEAM. Each agent (purple) owns one view (blue). Each arrow is a hand-off with one LLM-written value and its validator. The red validator checks only form, a weakness we meet in Section 10. The dashed agent is added while the team runs.

3.2 Reading team.yaml

The file `.agentm2m/team.yaml` describes the views (their classes, owner and seed model) and lists the hand-offs. Listing 2 shows the part for the Analyst's view.

Listing 2. Code excerpt: `templates/devteam/team.yaml` (abridged).

```
name: devteam
views:
  Req:
    owner: Analyst # the only agent that may edit this view
    classes:
      Epic: {attributes: [name]}
      Criterion: {attributes: [id, text]}
      UserStory:
        attributes: [id, title, status] # status: draft | accepted
        references:
          epic: Epic
          criteria: {type: Criterion, many: true, containment: true}
    root: {class: ReqModel, slots: {epics: Epic, stories: UserStory}}
    seed: # the initial model
    stories:
      - {id: S2, title: mark a task done, status: accepted, epic: "Epic#E1",
        criteria: [{id: S2.1, text: completing a done task is rejected}]}
      ...
    Arch: {owner: Architect, classes: {Operation: {attributes: [name, signature, criteria], ...}}, ...}
    ...
handoffs:
  - {name: Req2Arch, rule: rules/Req2Arch.agentm2m}
  - {name: Arch2Code, rule: rules/Arch2Code.agentm2m}
  - {name: Req2Test, rule: rules/Req2Test.agentm2m}
```

Every object has a *key* of the form `Type#id`, such as `UserStory#S2` or `Criterion#S2.1`; the tools use these keys to name objects. A reference such as `epic: "Epic#E1"` points to another object by its key. A *containment* reference (`criteria`) means the criteria are part of their story, like fields of an object.

3.3 Reading a rule module

Listing 3 shows the first hand-off in full, and Table 3 explains its notation. The expression language inside the rules is a small subset of OCL, the expression language of model-driven engineering; read `s.criteria` like a field access in Java.

Listing 3. Code excerpt: `templates/devteam/rules/Req2Arch.agentm2m`.

```
module Req2Arch;
create OUT : Arch from IN : Req;
uses 'helpers.py';

rule Epic2Component {
  from e : Req!Epic
  to c : Arch!Component ( name <- e.name )
}

rule Story2Operation {
  from s : Req!UserStory
    (s.status = #accepted)           -- guard: drafts are not handed off
  to op : Arch!Operation (
    name      <- s.id.toOpName(),    -- structural
    component <- s.epic,              -- resolved via trace (Epic -> Component)
    criteria  <- s.criteria.criteriaText(), -- structural copy, read later by Arch2Code
    signature <- @llm('Derive an API signature for this user story. Answer with exactly one
                      line of the form name(param: Type, ...) -> ReturnType and nothing else.',
                      s.criteria),    -- footprint
    @check signature.parses() and signature.params()->notEmpty() )
}
```

Table 3. The notation of a rule module.

Notation	Meaning
<code>module Req2Arch;</code>	The name of the hand-off.
<code>create OUT : Arch from IN : Req;</code>	It writes view Arch and reads view Req. A hand-off may read several views.
<code>uses 'helpers.py';</code>	Load Python helper functions from this file.
<code>rule R { from s : Req!UserStory</code>	For every object s of class UserStory in view Req ...
<code>(s.status = #accepted)</code>	...that satisfies this <i>guard</i> (<code>#accepted</code> is an enumeration literal) ...
<code>to op : Arch!Operation (...)</code>	...create one Operation op in view Arch, and fill its fields:
<code>name <- s.id.toOpName()</code>	a <i>structural binding</i> : the engine computes the value. <code>toOpName</code> is a Python function from <code>helpers.py</code> , called as if it were a method of <code>s.id</code> .
<code>component <- s.epic</code>	a <i>reference</i> : the engine replaces the source <code>epic</code> by the component it created from it (found through the trace).
<code>signature <- @llm(p, f)</code>	a <i>stochastic binding</i> : an LLM writes the value from prompt p and footprint f.
<code>@check e</code>	the validator of the binding just above it; e must be true.
<code>x->notEmpty()</code>	a collection operation (as in OCL): “x is not empty”.

Validators are ordinary Python functions in `helpers.py`. A good validator does not just return `False`; it returns a `Rejected(reason)` object, so that whoever tries again learns what was wrong (Listing 4).

Listing 4. Code excerpt: templates/devteam/rules/helpers.py (abridged).

```
def parses(signature: str):
    if signature_pares((signature or "").strip()):
        return True
    return Rejected("expected exactly one line: name(param: Type, ...) -> ReturnType")

def compiles(body: str):
    if python_compiles(_unfence(body)):
        return True
    return Rejected("not valid Python")
```

The other two hand-offs are similar. Arch2Code creates one CodeEdit per operation; its body is written by an LLM whose footprint is the operation's signature *and* the criteria that Req2Arch copied onto it, and whose validator is `compiles()`. Req2Test has no guard: it creates one TestCase per criterion, drafts included; its oracle is written from `c.text` and checked by `failsOnStub()`. That validator runs the oracle against a stub implementation that only raises `NotImplementedError` and accepts the oracle if it raises. It thus rejects an oracle that never calls the implementation; it does *not* reject one that calls it but asserts nothing, because the stub's exception already makes it fail.

3.4 The other two templates

Table 4 summarises all three templates. Two features of the others are worth knowing. The research template has a hand-off that reads *two* views in one rule (from `p : Experiments!ExperimentPlan, c : Literature!Claim (p.topic = c.topic)`), pairing every plan with every claim of the same topic. The incident template uses a *Lift* binding, `self <- @llm(...)`: the LLM answers with one JSON object, and the engine writes its keys onto several attributes at once, rejecting the answer unless every key names a real attribute. It also has an *executable-oracle* validator, `passesDryRun`, which runs an LLM-written Python script and accepts it only if it exits with code 0.

Table 4. The three templates of `team_init`.

Template	Agents (views)	What it teaches
devteam	Analyst (Req), Architect (Arch), Developer (Code), Tester (Test); Security Reviewer added at run time	Guards, blocked values, exact change impact, team evolution; a form-only validator on code.
research	Literature Reviewer, Experiment Designer, Report Writer	A multi-source hand-off (one rule reads two views).
incident	Monitor, Triage, Remediation, Postmortem	A Lift binding (one JSON answer fills several fields); an executable-oracle validator that runs LLM-written scripts.

Q2. Check your understanding. In `DEVTEAM`, `Story20peration` will match 2 stories but `Criterion2TestCase` 3 criteria. Using Listing 3 and the description of `Req2Test`, explain the difference.

Key takeaways.

- `team.yaml` declares views (classes, owner, seed) and hand-offs; objects are named by keys such as `Criterion#S2.1`.
- A rule says: for every source object matching the pattern and guard, create one target object; structural bindings are computed, `@llm` bindings are written by an LLM from their footprint, `@check` decides.

- Validators are Python functions that return `True` or `Rejected(reason)`.

4 The problem: from a research library to a tool

4.1 Where we started

Before version 0.2.0, AGENTM2M could be used only as a Python library. To run a team you wrote Python that built metamodels, created seed models, registered hand-offs and called the runtime (the files `examples/*/run.py` still show this). All state lived in memory and vanished when the program ended. The engine called an LLM itself, through an API key for OpenAI or Anthropic or through a local Ollama server (a program that runs open LLMs on your own machine). None of this fits how a developer works: in an editor, with an assistant, over days, on a project that changes.

4.2 Seven requirements

We phrase the problem as requirements, as you would in a requirements engineering course. Table 5 says where the design answers each, and Section 15 checks how well it did.

R1 Usable from the assistant.

A developer must be able to start, run, change and inspect a team by talking to Claude Code, without writing Python.

R2 No second LLM bill.

The developer already pays for Claude Code's LLM. Requiring a second API key so that the engine can call an LLM would be a barrier. Yet the engine's guarantees (validators, escalation, stamps, φ) must not weaken.

R3 Footprint discipline survives.

Change impact is exact only if each value was written from its footprint and nothing else. An assistant that has read the whole repository could quietly use more. The design must make the footprint the *only* information available, not just ask politely.

R4 State survives.

Models, trace links and stamps must persist across sessions and survive several processes touching them.

R5 Mistakes are caught early.

When the assistant edits a rule file or a typed team, errors must be reported immediately, before anything runs.

R6 Usable without installing the engine.

Teams that share a server, or users who cannot install Python packages, should be able to use the same tools remotely.

R7 Safe.

The engine executes code: helpers that the project provides, and code that the LLM writes. A shared server must not let one user's code see another's data.

Key takeaways.

- Turning the engine into a tool is a software engineering problem with explicit requirements, not just packaging.
- The two hardest requirements pull against each other: use Claude Code's LLM (R2) without letting Claude Code's broad knowledge leak into values (R3).

Table 5. Requirements and where the design answers them.

Req.	Design answer	Section
R1	An MCP server exposes 24 tools; skills script the common workflows.	5, 9, A
R2	<i>Host mode</i> : the engine never samples; Claude Code answers engine-built prompts, and the engine judges the answers exactly as its own samples.	8
R3	Subagents whose tool allowlist contains only “get my next prompt” and “submit my answer”; values tied to a <code>footprint_version</code> .	8, 9
R4	A workspace directory <code>.agentm2m/</code> with atomic writes, a file lock and reload-on-change.	7
R5	A hook re-validates after every edit of a team file and feeds errors back to Claude Code.	9
R6	<code>agentm2m serve</code> : the same tools over HTTP, API keys, per-key projects.	12
R7	Write rights, path checks, one sandbox function, and a service that refuses to execute code without an isolating sandbox.	13

5 The Model Context Protocol (MCP)

5.1 The problem MCP solves

Suppose there are N AI assistants and M tools (a database, an issue tracker, AGENTM2M). If every assistant integrates every tool in its own way, someone writes $N \times M$ adapters. If all speak one protocol, each side writes one adapter: $N + M$. The Language Server Protocol [10] did this for editors and programming languages: any editor can use any language server. The *Model Context Protocol* (MCP) [11, 12] does it for AI assistants: it standardises how an assistant discovers what a tool server offers and how it calls it. It is built on JSON-RPC 2.0.

5.2 Roles: host, client, server

MCP names three roles (Figure 3). The **host** is the application the user works in; here, Claude Code. Inside the host, one **client** manages the connection to one server. The **server** is the program that offers capabilities; here, the program `agentm2m-mcp` (`src/agentm2m/mcp_server.py`). The LLM in the host sees the tools of all connected servers.

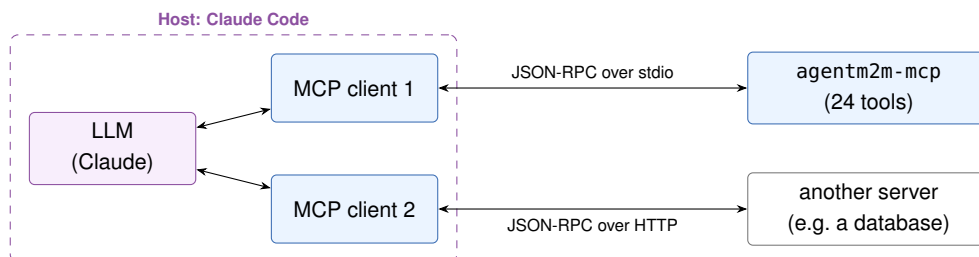


Figure 3. MCP roles. The host contains one client per server. The LLM decides which tool to call; the host performs the call through the right client.

5.3 What a server can offer

MCP defines three kinds of capabilities, called *primitives*. **Tools** are functions the LLM may decide to call, such as `run` or `submit_binding`. **Resources** are pieces of data the host may read, such as a file. **Prompts** are reusable prompt templates the user may pick. A server also sends free-text **instructions** when the connection opens, telling the LLM how to use it. The AGENTM2M server uses tools and instructions only (the SDK advertises empty resource and prompt lists). Its instructions summarise the intended order of calls and the one rule that matters most: “Answer a binding ONLY from the prompt next_bindings returns”.

5.4 The life of a connection

A connection goes through four steps. We captured them by starting the real server and writing JSON lines into its stdin (script `mcp_stdio_demo.py`).

1. Initialize. The client says which protocol version it speaks and what it can do. The server answers with the version both will use (here it accepts the client's), its capabilities and its instructions.

Listing 5. Observed: the initialize handshake with `agentm2m-mcp` over stdio (instructions abridged).

```
-> {"jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {"protocolVersion": "2025-06-18",
  "capabilities": {}, "clientInfo": {"name": "demo", "version": "1"}}}
<- {"jsonrpc": "2.0", "id": 1, "result": {"protocolVersion": "2025-06-18",
  "capabilities": {"prompts": {"listChanged": false}, "resources": {"listChanged": false, "subscribe": false},
    "tools": {"listChanged": false}},
  "serverInfo": {"name": "agentm2m", "version": ""},
  "instructions": "agentm2m runs a team of agents whose hand-offs are model-to-model
    transformations. ... Typical loop: team_status -> run -> next_bindings ->
    submit_binding (repeat) -> acceptance. ... Answer a binding ONLY from the prompt
    next_bindings returns: it already contains the whole allowed footprint. ..."}
-> {"jsonrpc": "2.0", "method": "notifications/initialized"}
```

(The empty `serverInfo.version` is harmless: the server does not tell the SDK its version. `agentm2m-mcp-version` prints it.)

2. Initialized. The client confirms with a *notification*. Only now may normal requests flow.

3. Discover. The client asks `tools/list`. For each tool, the server returns its name, a description and a JSON Schema of its arguments (Listing 6).

Listing 6. Observed: the `tools/list` entry for `submit_binding`, one of 24.

```
{"name": "submit_binding",
  "description": "Host mode: submit a value for one pending binding, passing the footprint_version
    that came with its prompt. Status is accepted, rejected (with reason and a retry_prompt;
    resubmit), escalated (validator kept rejecting: needs a source change or a human),
    already_accepted, or stale (the footprint changed since the prompt was issued: fetch it again).",
  "inputSchema": {"type": "object", "title": "submit_bindingArguments",
    "properties": {"target_key": {"title": "Target Key", "type": "string"},
      "binding": {"title": "Binding", "type": "string"},
      "value": {"title": "Value", "type": "string"},
      "footprint_version": {"title": "Footprint Version", "type": "string"}},
    "required": ["target_key", "binding", "value", "footprint_version"]}}
```

The description is the *only* documentation the LLM gets about a tool, so it is written for the LLM: it names every outcome and what to do next. Nobody wrote the schema by hand. The Python MCP SDK derives it from the function's type hints, and the description from its docstring (Listing 7); change the signature and the schema follows. The `@mcp.tool()` line is a Python *decorator*: a function that wraps the function below it, here to register it as a tool.

Listing 7. Code excerpt: how a tool is declared (`src/agentm2m/mcp_server.py`).

```
@mcp.tool()
@_forwardable
def submit_binding(target_key: str, binding: str, value: str, footprint_version: str,
    ctx: Context | None = None) -> dict:
    """Host mode: submit a value for one pending binding, passing the
    footprint_version that came with its prompt. Status is accepted, rejected ..."""
    return _call(_ws(ctx).submit_binding, target_key, binding, value, footprint_version)
```

The SDK fills the parameter `ctx` with information about the current request (for example HTTP headers) and leaves it out of the schema. `@_forwardable` is a second decorator, explained in Section 12.6.

4. Call. The LLM decides to call a tool; the client sends `tools/call` with the name and arguments; the server runs the function and returns its result as text content (Listing 8).

Listing 8. Observed: a successful call (JSON text inside content abridged).

```
-> {"jsonrpc": "2.0", "id": 3, "method": "tools/call",
    "params": {"name": "team_init", "arguments": {"template": "devteam"}}}
<- {"jsonrpc": "2.0", "id": 3, "result": {"isError": false, "content": [{"type": "text",
    "text": "\n  \team\":  \devteam\", \n  \backend\":  \host\", \n  \agents\": {...} ..."}]}}
```

5.5 Two kinds of errors

Listing 9 shows three failed requests side by side.

Listing 9. Observed: a protocol error and two tool errors.

```
-> {"jsonrpc": "2.0", "id": 2, "method": "tools/run", "params": {}}
<- {"jsonrpc": "2.0", "id": 2, "error": {"code": -32601, "message": "Method not found", "data": "tools/run"}}

-> {"jsonrpc": "2.0", "id": 3, "method": "tools/call", "params": {"name": "no_such_tool", "arguments": {}}}
<- {"jsonrpc": "2.0", "id": 3, "result": {"isError": true, "content": [{"type": "text",
    "text": "Unknown tool: no_such_tool"}]}}

-> {"jsonrpc": "2.0", "id": 4, "method": "tools/call", "params": {"name": "impact", "arguments":
    {"view": "Req", "as_agent": "Nobody", "ops": [{"op": "delete", "key": "UserStory#S3"}]}}}
<- {"jsonrpc": "2.0", "id": 4, "result": {"isError": true, "content": [{"type": "text",
    "text": "Error executing tool impact: unknown agent 'Nobody'; agents: ['Analyst',
    'Architect', 'Developer', 'Tester']"}]}}
```

The first is a *protocol error*: `tools/run` is not an MCP method, so the answer is a JSON-RPC error. That is a bug in a program and goes to the host. The other two are *tool errors*: the protocol worked, the *tool* failed, which the server signals with a normal result that has `"isError": true` and a readable message. A tool error is information for the LLM, which can read it and correct itself, here by naming a real agent. The AGENTM2M server turns every expected failure into such a sentence (Listing 10). Unexpected exceptions are reported with their Python type name; that reveals a little about the internals, which is acceptable for a tool that runs on your own machine or behind an API key.

Listing 10. Code excerpt: expected failures become readable tool errors (`mcp_server.py`).

```
def _call(fn, *args, **kwargs) -> dict:
    try:
        return fn(*args, **kwargs)
    except (WorkspaceError, SandboxRefused) as exc:
        # A deliberate, actionable tool error (shown to the model verbatim),
        # not a crash with a traceback.
        raise ToolError(str(exc)) from None
    except Exception as exc: # e.g. an OCL error in a user's rule module
        raise ToolError(f"{type(exc).__name__}: {exc}") from exc
```

5.6 Transports: stdio and Streamable HTTP

MCP messages travel one of two ways.

- **stdio.** The host starts the server as a child process and exchanges one JSON message per line over its stdin and stdout. Nothing goes over the network; the server lives as long as the host session. This is how the plugin runs by default.
- **Streamable HTTP.** The server is a web service with one endpoint (here `/mcp`). The client sends every message as an HTTP POST; the server answers with plain JSON or with a short stream of *server-sent*

events (a way for one HTTP response to deliver several messages one after another). The server may assign a session id in the header `Mcp-Session-Id` when the connection is initialised; the `AGENTM2M` service does, and the client then repeats it on every request, together with `MCP-Protocol-Version`. (The specification also uses `GET` to open a stream from the server and `DELETE` to end a session.) Authentication is ordinary HTTP, here an API key in the `Authorization` header. The hosted service uses this transport (Section 12).

5.7 Tool names inside Claude Code, and things called “agentm2m”

Claude Code gives every MCP tool a globally unique name, so that tools of different servers cannot clash: `mcp__<server>__<tool>`. A server that comes with a plugin also carries the plugin’s name. Many things in this project are called `agentm2m`, which makes the resulting names look odd; Table 6 sorts them out.

Table 6. Things called `agentm2m`.

Name	What it is
<code>agentm2m</code> (Python package)	The engine, version 0.3.0.
<code>agentm2m</code> (command)	Its command-line interface: <code>agentm2m workspace ...</code> , <code>agentm2m auto ...</code> , <code>agentm2m serve</code> .
<code>agentm2m-mcp</code> (command)	The program that runs the MCP server over stdio.
<code>agentm2m</code> (MCP server name)	The name under which the plugin’s <code>.mcp.json</code> registers that server.
<code>agentm2m</code> (plugin)	The Claude Code plugin in <code>plugin/agentm2m/</code> ; its skills are <code>/agentm2m:<skill></code> .
<code>agentm2m-dev</code> (marketplace)	The development marketplace in <code>plugin/.claude-plugin/</code> .

So `submit_binding` appears to Claude Code as `mcp__plugin_agentm2m_agentm2m__submit_binding`: the prefix `mcp`, then `plugin_` + plugin name `agentm2m` + `_` + server name `agentm2m`, then the tool name, joined by double underscores. If you connect the hosted service directly, without the plugin (Section 12), the same tool is `mcp__agentm2m__submit_binding`. These names matter because subagent allowlists (Section 9) must spell them exactly.

Remark 3 (What MCP does and does not give you). MCP makes a tool *discoverable* and *callable* with well-formed arguments. It says nothing about how a *team* of agents fits together: who owns which artefact, whether every requirement reaches a test, who decides that the work is done. The `AutoM2M` teaching edition lists MCP among the approaches that make individual messages well formed but check no team-level property [1]. Here MCP is the *transport*; the team-level guarantees come from the engine behind it.

Q3. Check your understanding. The server’s instructions and each tool’s description are plain text read by the LLM, not by a program. What could go wrong if a tool’s description said “returns the accepted value” while the tool actually returns a status object? Who would notice?

Key takeaways.

- MCP standardises how a host discovers and calls the tools of a server, over JSON-RPC 2.0: `initialize`, `initialized`, `tools/list`, `tools/call`.
- Tool schemas are generated from Python type hints; descriptions are written for the LLM and name every outcome.
- Tool failures are results with `isError: true` that the LLM can act on; protocol errors are JSON-RPC errors, i.e. bugs.
- stdio for a local server; Streamable HTTP with an API key for a remote one.

6 Claude Code plugins

A Claude Code plugin is a directory with a fixed layout [3]. Listing 11 shows the AGENTM2M plugin.

Listing 11. Code excerpt: the layout of plugin/agentm2m/, version 0.3.0.

```
plugin/agentm2m/
.claude-plugin/plugin.json    manifest: name, version, description, license, keywords
.mcp.json                    which MCP server(s) to start, and how
skills/<name>/SKILL.md        10 skills: init, run, change, evolve, status, author-handoff,
                                auto-build, check, diagnose, agentm2m-concepts
agents/<name>.md              4 subagents: binding-worker, handoff-architect,
                                team-builder, auto-binding-worker
hooks/hooks.json             2 hooks: SessionStart, PostToolUse (Edit|Write|MultiEdit)
hooks/session-start.sh, hooks/validate-on-edit.sh
README.md, CHANGELOG.md, LICENSE
plugin/.claude-plugin/marketplace.json  a development marketplace listing this plugin
```

(Plugins may contain more, for example commands/; this one does not.)

6.1 The manifest

plugin.json names the plugin and its version. The name (agentm2m) becomes the namespace of everything the plugin adds: its skills are invoked as /agentm2m:run, its tools are prefixed mcp__plugin__agentm2m_... The version, 0.3.0, matches the engine's; Section 14 explains why.

6.2 The MCP server configuration

.mcp.json tells Claude Code how to start the server (Listing 12).

Listing 12. Code excerpt: plugin/agentm2m/.mcp.json.

```
{"agentm2m": {
  "command": "uvx",
  "args": [ "--from", "${AGENTM2M_ENGINE:-agentm2m==0.3.0}", "--with", "mcp>=1.2", "agentm2m-mcp"],
  "env": { "AGENTM2M_LLM": "${AGENTM2M_LLM:-host}",
    "AGENTM2M_URL": "${AGENTM2M_URL:-}",
    "AGENTM2M_API_KEY": "${AGENTM2M_API_KEY:-}",
    "AGENTM2M_PROJECT": "${AGENTM2M_PROJECT:-}" }}
```

Read it in three parts.

- uvx is a command of the Python package manager uv [13]. uvx -from P C installs package P into a cached, isolated environment (once) and runs its command C. The plugin therefore contains no engine code: the engine is an ordinary Python package, and -with mcp>=1.2 adds the MCP SDK to its environment.
- \${AGENTM2M_ENGINE:-agentm2m==0.3.0} is shell-style substitution: “the value of the environment variable AGENTM2M_ENGINE, or agentm2m==0.3.0 if it is unset”. The default will fetch version 0.3.0 from the Python Package Index once it is published; until then (see Section 14) you must set AGENTM2M_ENGINE to the path of a checkout of the repository.
- env passes four settings to the server: who fills values (AGENTM2M_LLM, default host, Section 8) and, if set, the URL, API key and project of a hosted service (Section 12).

6.3 Skills

A *skill* is a Markdown file of instructions with a short YAML header (*frontmatter*). The user can invoke it as a slash command (/agentm2m: run), and Claude Code also loads it by itself when the skill's description matches the task. Whatever the user types after the command replaces the placeholder \$ARGUMENTS. A skill is a *procedure for the LLM*, in plain language: which tools to call, in which order, what to check, what to report (Listing 13).

Listing 13. Code excerpt: the start of skills/change/SKILL.md.

```
---
name: change
description: Apply a change to an agentm2m team's source view (e.g. a tightened acceptance
  criterion or a new user story), preview exactly which downstream artifacts it obliges to be
  redone, then propagate it. Use when requirements or any upstream artifact of the team changes.
argument-hint: "<describe the change>"
---
# Change and propagate
1. Call `team_status` and `model_show` for the view to change. Identify the owning agent ...
2. Turn `$ARGUMENTS` into `model_edit` ops (`create` / `set` / `delete`, element keys like
  `Criterion#S2.1`). Do not apply them yet.
3. Call `impact` with `view`, `ops`, and `as_agent` to preview. Show the user: ...
```

argument-hint is what Claude Code shows when you start typing the command. One skill, agentm2m-concepts, sets user-invocable: false: it is not a command but background knowledge (“AGENTM2M in one page”) that Claude Code loads by itself in projects with a .agentm2m/ directory.

6.4 Subagents

A file in agents/ defines a subagent: a name, a description (when to use it), a model (inherit: the same model as the main agent) and a tools *allowlist*. A subagent can call only the tools on its list. Section 9 shows how the plugin uses this.

6.5 Hooks

A *hook* is a shell command that Claude Code runs automatically at a fixed moment, an *event*. The plugin registers two (Listing 14): one when a session starts, and one after every use of the Edit, Write or MultiEdit tools. \${CLAUDE_PLUGIN_ROOT} is replaced by the directory where the plugin is installed. A hook talks back through its output and exit code: for SessionStart, what it prints is added to Claude Code's context; for PostToolUse, exit code 2 means “something is wrong”, and what it wrote to stderr is shown to Claude Code, which can then fix it. Note that edits made through Bash (for example with sed) do not match the hook.

Listing 14. Code excerpt: hooks/hooks.json (abridged).

```
{
  "hooks": {
    "SessionStart": [
      {
        "type": "command",
        "timeout": 10,
        "command": "bash \"${CLAUDE_PLUGIN_ROOT}/hooks/session-start.sh\""
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "timeout": 120,
            "command": "bash \"${CLAUDE_PLUGIN_ROOT}/hooks/validate-on-edit.sh\""
          }
        ]
      }
    ]
  }
}
```

6.6 Marketplaces and installation

Plugins are distributed through *marketplaces*: a file `.claude-plugin/marketplace.json` that lists plugins and where their directories are. A user adds a marketplace once and installs plugins from it as `plugin@marketplace`. The repository ships a development marketplace, `agentm2m-dev`; the script `plugin/scripts/dev-install.sh` registers it and installs the plugin from your checkout. Appendix B gives the complete steps to try it.

Table 7. Claude Code’s extension points and what the AGENTM2M plugin puts in each.

Extension point	What it is	In the AGENTM2M plugin
MCP server	Tools the LLM can call	<code>agentm2m-mcp</code> : 24 tools (12 for hand-written teams, 12 for AUTO2M2M)
Skills	Procedures in plain language	10 skills, 9 of them slash commands
Subagents	Helpers with their own tool allowlists	4 subagents; two of them can only fetch and submit values
Hooks	Commands run at fixed events	announce a team at session start; re-validate after edits
(Distribution)	Marketplace	<code>agentm2m-dev</code> (local); a public one at release

Q4. Check your understanding. Which extension point would you use for each of these? (a) After every edit of a rule file, check that it still parses. (b) Teach Claude Code the steps of propagating a requirement change. (c) Guarantee that the helper which writes test oracles cannot open the project’s source files. (d) Let the LLM preview the impact of a change.

Key takeaways.

- A plugin is a directory: a manifest, an MCP server configuration, skills, subagents and hooks; the engine itself is a separate Python package.
- `uvx` installs the pinned engine and runs its MCP server; `AGENTM2M_ENGINE` points it at a local checkout instead.
- Skills are procedures for the LLM; subagents have tool allowlists; hooks run at fixed events and report back through exit code 2.

7 Architecture of the plugin

7.1 Layers

Figure 4 shows the layers from the user down to the files on disk. Each layer talks only to the one below it: a *layered architecture*, as in networking or web applications.

7.2 The MCP layer is deliberately thin

Every tool function in `mcp_server.py` is one or two lines: find the workspace, call one method, return its dictionary (Listing 7). All logic lives in the *service layer*: `Workspace` (`src/agentm2m/workspace.py`) for hand-written teams and `AutoWorkspace` (`src/agentm2m/auto/workspace.py`) for AUTO2M2M. Three benefits follow, as in any layered design. *Testability*: the service layer is tested with plain Python calls, without an MCP server; most of the plugin’s tests do exactly that. *Reuse*: the command-line interface, the MCP server and the REST API of the hosted service all call the same methods, so they cannot behave differently. *Extensibility*: the hosted service added a REST API without touching any business logic; each endpoint calls the same method as the corresponding tool.

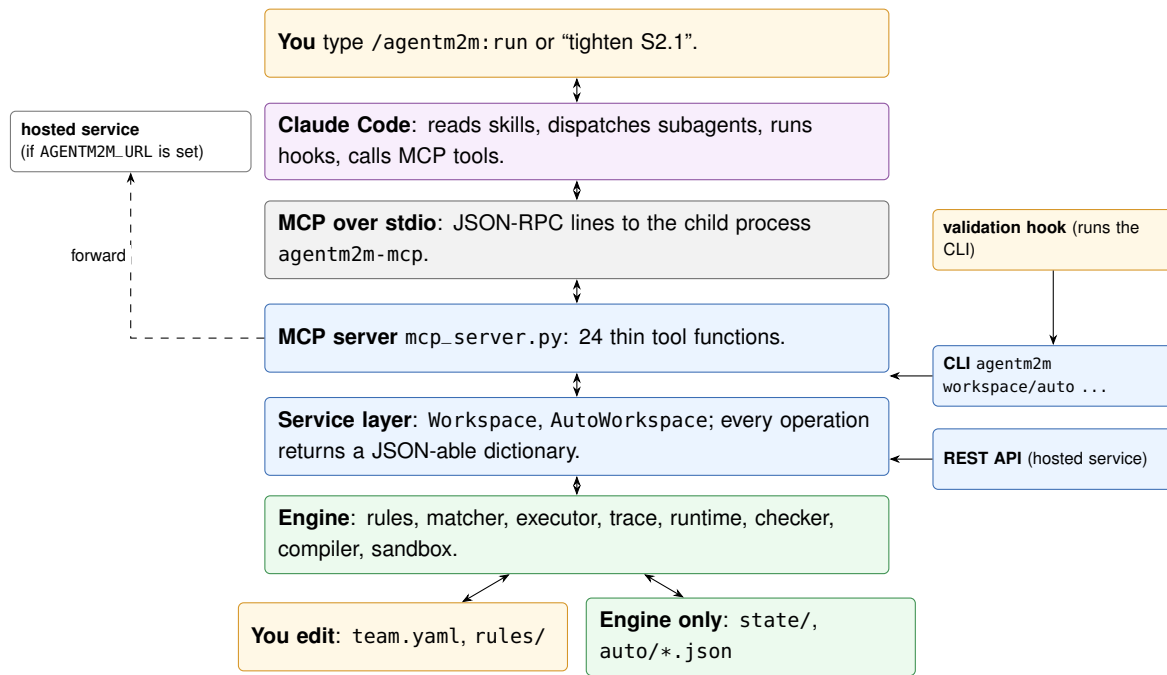


Figure 4. The layers of the plugin. Only Claude Code involves an LLM; everything from the MCP server down is deterministic. The CLI, the validation hook and the REST API of the hosted service reach the same service layer.

7.3 Which project does a tool act on?

A tool call carries no project path. The server finds the project directory by a fixed rule: on the hosted service, from the authenticated user and the requested project (Section 12); locally, from `AGENTM2M_PROJECT_DIR`, else `CLAUDE_PROJECT_DIR` (the project Claude Code was started in), else the current directory, which for a server started by Claude Code is the project. One `Workspace` object is kept per directory.

7.4 What is stored where

Table 8. The workspace directory `.agentm2m/` of a project.

File	Contents	Written by
<code>team.yaml</code>	views and hand-offs (Listing 2)	you
<code>rules/*.agentm2m</code>	rule modules (Listing 3)	you
<code>rules/helpers.py</code>	validators and helpers; <i>executed</i> by the engine	you
<code>state/state.json</code>	view models, trace links, stamps, team evolutions	engine
<code>state/.lock</code>	the file lock	engine
<code>auto/task.json</code>	AutoM2M: the lifted task	engine
<code>auto/team.json</code>	AutoM2M: the admitted typed team	engine (via <code>auto_submit_team</code>)
<code>auto/proposal.json</code>	AutoM2M: the last rejected proposal	engine
<code>auto/state.json</code>	AutoM2M: models and trace links of the running team	engine
<code>auto/history.json</code>	AutoM2M: admissions, runs, attributions (last 200 events)	engine
<code>auto/faults.json</code>	AutoM2M: the last fault reports	engine
<code>auto/compiled/, auto/runs/</code>	AutoM2M: generated rule modules; work files of unattended runs	engine

Commit `team.yaml` and `rules/` to version control: you wrote them. Leave `state/` and `auto/` to the engine (commit them only if you want to share a run). The session-start hook repeats the warning “never edit `state/` by hand”: a hand edit could give a value a stamp it never earned.

7.5 Several processes, one directory

The same workspace can be touched by several processes: the MCP server, the validation hook (which runs the CLI in a separate process), a second Claude Code session, or you on the command line. Three mechanisms protect the files.

1. **Atomic writes.** State is written to `state.json.tmp` and then renamed over `state.json` with `os.replace`. On POSIX file systems (Linux, macOS) a rename is atomic: a reader sees the old file or the new one, never half of one.
2. **A file lock.** Each load and each write of a hand-written team's state takes an exclusive operating-system lock on `state/.lock` (`fcntl.flock`), so no process reads a file while another writes it. Inside one process, a re-entrant lock (`threading.RLock`, a lock the same thread may take again) makes concurrent tool calls run one at a time. The two locks answer different questions: one is between processes, the other between threads.
3. **Reload on change.** Before each operation the workspace computes a *fingerprint*: modification time and size of `team.yaml`, `state.json` and every rule file. If it differs from the one remembered after the last load, another process changed something, and the workspace reloads from disk.

These mechanisms keep the *files* consistent. They do not make a whole operation atomic across processes, as this interleaving shows (we ran it with two Workspace objects on one directory):

Time	Process A	Process B
1	loads the state	
2		submits the signature of S2, saves
3	submits the signature of S1: accepted	
4	saves: notices the fingerprint changed, reloads, writes	
result	S2 fresh; S1 back to <i>missing</i> although A reported accepted	

The state stays consistent, and the lost value is simply offered again by the next `next_bindings`; but an acceptance that was reported can be lost. This is the classic *lost update* problem of concurrent programming. Within one server process it cannot happen (the re-entrant lock serialises every operation); across processes it can. `AUTO2M2M`'s files have atomic writes and reload-on-change, but no file lock.

Remark 4 (Errors leave no trace). `model_edit` is atomic in the database sense: it takes a snapshot of the state, applies all operations, and restores the snapshot if any of them fails. Either every operation of an edit is applied, or none is.

Q5. Check your understanding. In the interleaving above, why does process A's save not simply overwrite B's change? And why is it acceptable that A's value is lost?

Key takeaways.

- Layers: Claude Code → MCP (stdio) → thin tool functions → service layer → engine → files. Only Claude Code uses an LLM.
- The service layer is shared by MCP, CLI and REST, so all three behave the same and can be tested without a server.
- Atomic renames, a file lock and fingerprint reloads keep the files consistent; overlapping operations of two processes can still lose a reported acceptance, which is then re-offered.

8 Host mode: Claude Code plays the agents, the engine decides

8.1 Who calls the LLM?

In the original library the engine called an LLM through a *backend*: a small class with one method, `generate(prompt) -> str`. The plugin keeps those backends and adds one (Table 9). The environment variable `AGENTM2M_LLM` chooses.

Table 9. Backends: who produces the values of `@llm` bindings.

AGENTM2M_LLM	Who fills the values
host (default)	Claude Code, through <code>next_bindings</code> and <code>submit_binding</code> . No extra API key.
anthropic, openai, ollama	The engine calls that provider itself (an <i>engine LLM</i> ; needs an API key, or a local Ollama). Good for unattended runs.
mock	Canned values, for demos and tests.

8.2 The trick: a backend that never answers

How can the engine run its normal algorithm (match, create, bind, sample, validate, stamp) without sampling? The host backend is marked *deferred*. When the engine reaches a value that needs (re)sampling, it checks this mark, builds the complete prompt itself, and records a `PendingBinding` instead of asking for a sample. (The host backend's `generate` method throws an exception, `PendingSample`, in case any code path ever calls it anyway.) The prompt is the binding's prompt text, followed by "Context (footprint only):" and the footprint rendered as text. Listing 15 shows a real pending value as `next_bindings` returns it.

Listing 15. Observed: one pending value, as returned by `next_bindings` (from Section 10, Step 8).

```
{
  "handoff": "Arch2Sec", "rule": "Operation2Review",
  "target_key": "Operation2Review::sr::op=Operation#op_s1",
  "binding": "notes",
  "prompt": "Summarize any authentication or authorization concerns for this operation signature\nin at most three sentences.\n\nContext (footprint only):\ncreate_task(title: str) -> Task",
  "attempts": 0, "kind": "new", "footprint_version": "216262ec9373d8d9",
  "agent": "SecurityReviewer", "max_attempts": 3
}
```

- `handoff`, `rule`, `binding`: where the value comes from.
- `target_key`: the value's place, `Rule::variable::match`; read "rule `Operation2Review`, target variable `sr`, for the match `op = Operation#op_s1`".
- `prompt`: everything the writer of the value may know.
- `attempts`, `max_attempts`: rejections so far, and k .
- `kind`: `new` (never accepted) or `stale` (accepted once, footprint changed since).
- `footprint_version`: the digest of the footprint, which becomes the stamp if the value is accepted.
- `agent`: the owner of the target view, who should write it.

The structural part of each hand-off runs normally: every target object, reference and trace link exists before any value is asked for.

In plain words: *the engine writes the question and keeps the answer key; Claude Code only fills in the blank.*

Q6. Check your understanding. Why can't the host backend simply call Claude Code's LLM from inside `generate()` and wait for the answer? Think about who calls whom when a tool runs.

8.3 One value, from prompt to stamp

Figure 5 follows one value through the system.

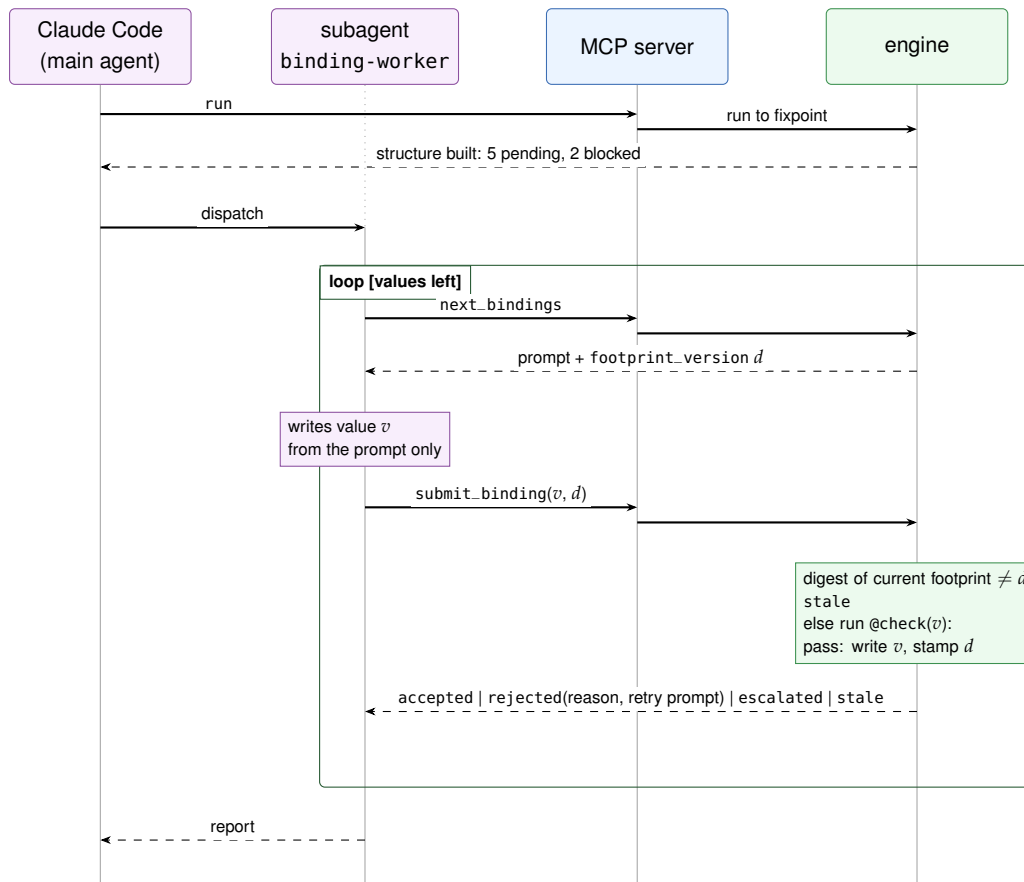


Figure 5. Host mode for one value. Claude Code and its subagent (purple) only write text from a prompt; the server (blue) and the engine (green) decide everything else. The subagent exists only from its dispatch on, and can call only the two tools shown. Return messages pass through the server.

When a value is submitted, the engine does exactly what it would do with a sample it drew itself, through the same function (`accept_sample`): it runs the `@check` validator, and only if it passes writes the value into the target object and records the stamp. Validators, the budget k , escalation, stamps and φ are therefore identical with and without an engine LLM. This is how the design meets R2.

8.4 The five answers to a submission

Table 10. The statuses `submit_binding` returns, and what the caller should do.

Status	Meaning	Next step
accepted	The validator passed; the value is written and stamped.	Next value.
rejected	The validator failed. The answer carries the reason, <code>attempts_left</code> and a <code>retry_prompt</code> .	Fix exactly what the reason says; resubmit.
escalated	The k -th rejection on the same footprint (or an earlier escalation). The value is not offered again for this footprint.	Leave it: it needs a change of the source or the team, or a human.
stale	The footprint changed since the prompt was issued, or the target no longer exists.	Fetch the value again; it comes back with a new prompt.
already_accepted	A value is already accepted for this footprint.	Skip it.

The `retry_prompt` is the original prompt plus “Your previous answer was rejected (*reason*): *your answer* (first 400 characters). Answer again . . .”. It adds only the model’s *own* previous answer and the validator’s reason, never source data outside the footprint. The code’s comments explain why it exists: re-sending the identical prompt at low temperature tends to reproduce the same rejected answer.

8.5 Why footprint_version exists

The development log (`plugin/DEVELOPMENT_PLAN.md`) records a bug found during live testing: “a code body written against the old signature was stamped against the new one”. The following reconstruction shows how that can happen. A Developer subagent receives the prompt for a code body, which contains the operation’s signature. While it writes, the Architect’s new signature is accepted. The Developer then submits code written against the *old* signature, and the engine stamps it with the digest of the *new* footprint, as if the code had been derived from data it never saw. Change impact is then silently wrong. The fix: every prompt carries the digest of its footprint, the submission must return it, and the engine refuses a value whose digest no longer matches (*stale*). This is *optimistic concurrency control* [14], the idea behind a version number on a database row or the If-Match header in HTTP [5]: work without locking, and check at the end that nothing changed underneath.

In plain words: *footprint_version* is a receipt that says which version of the inputs you answered; an answer with an old receipt is sent back.

8.6 Blocked values

Some footprints read values that an LLM writes upstream: the Developer’s code body reads the Architect’s signature. With an engine LLM, hand-offs run in order and the signature always exists in time. In host mode values arrive whenever Claude Code submits them, so the engine could offer the Developer a prompt with an empty signature. It does not: a value whose footprint reads an upstream value that has *never* been accepted is *blocked* and offered only once that value is accepted. A *stale* upstream value does not block: it exists, so it is shown, and if it is later replaced, the *footprint_version* check catches the outdated answer (Section 10, Step 6). That is why the skills say “call run again: accepted upstream values unblock downstream bindings”.

8.7 The life of one value

Figure 6 summarises the states of one value as `team_status` reports them.

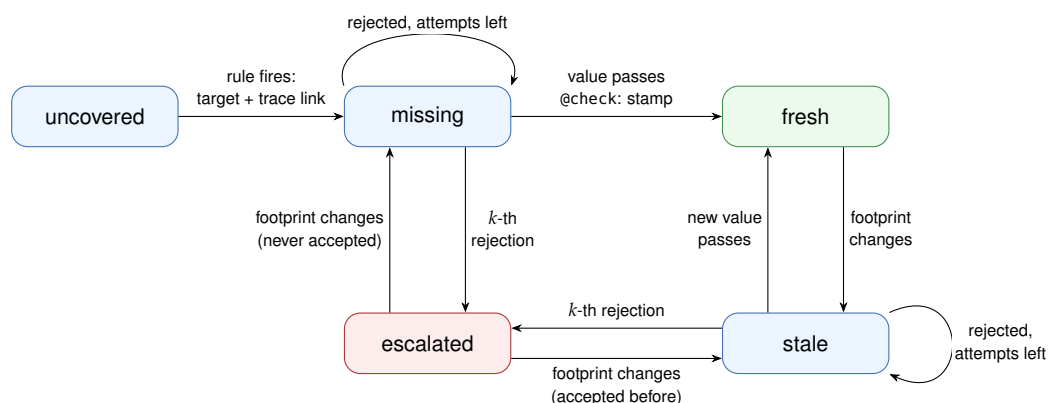


Figure 6. States of one value for a hand-written team. “Missing” and “stale” values are *pending*: offered by `next_bindings`, unless they are *blocked* (Section 8.6). An escalated value gets a fresh budget only when its footprint changes. For an AUTO2M2M team, an admitted repair also clears every escalation (Section 11).

8.8 What host mode keeps, and what it gives up

Kept. Every structural decision (which objects exist, how they link, which trace links are recorded) is made by the engine. Every value passes the same validators, under the same budget k , and gets the same stamp. “Done” is still φ , computed by the engine.

Given up. The engine no longer controls *which* model writes a value, at what temperature, or what else that model has seen. When the engine calls an LLM provider itself, that LLM receives the prompt and nothing else. Claude Code, in contrast, has seen the whole conversation. Footprint discipline therefore has to be enforced from the outside, by the plugin (Section 9).

Q7. Check your understanding. The Developer subagent fetched the prompt for the code body of `op_s2`. Before it submitted, the Analyst changed criterion `S2.1`, and the Architect’s new signature for `op_s2` was accepted. The Developer now submits its code with the old `footprint_version`. Which status does it get, and why is that right?

Key takeaways.

- In host mode the engine never samples: it builds the complete, footprint-bounded prompt and waits for Claude Code to submit a value.
- A submitted value goes through exactly the same acceptance path as an engine sample: validator, budget k , escalation, stamp.
- `footprint_version` refuses answers to outdated prompts; values whose upstream input was never accepted are blocked.
- What the engine cannot control (which model, what else it saw) the plugin must control with subagents.

9 Skills, subagents and hooks

The tools are the *what*; skills, subagents and hooks are the *how*. This section shows how they cooperate, and how they close the gap host mode leaves open (R3) and catch mistakes early (R5).

9.1 The skills

Two habits run through Table 11. The skills that change a team *preview* or *check first*: `impact` before `model_edit` in `/agentm2m:change`, the checker inside `auto_submit_team` before anything runs in `/agentm2m:auto-build`. And reports are tied to tool results; `/agentm2m:change` even says “Name only elements that appear in tool results”, which guards against the LLM inventing a plausible but false summary.

9.2 The subagents and least privilege

Table 12 lists the four subagents and their tool allowlists (tool names abbreviated; in the files they are the full names of Table 6).

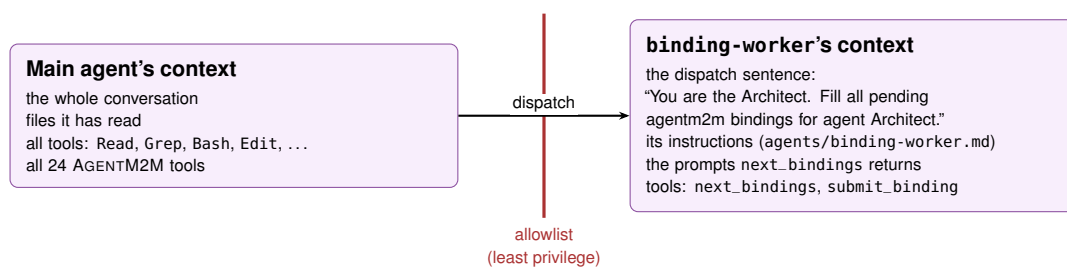
The two binding workers are the plugin’s answer to R3 (Figure 7). A binding-worker cannot read a file, search the repository or run a command: those tools are not on its list, so Claude Code will not let it call them. It starts with a fresh context, and its only source of facts about the team is the prompt `next_bindings` returns, which is the footprint. By the principle of least privilege, the worker cannot *fetch* anything beyond its prompts. What it is *told* at dispatch is still written by the main agent, which has seen everything; the skill therefore fixes that message to one sentence. The guarantee is strong but not absolute (Section 15).

Table 11. The ten skills of the plugin.

Skill	What it makes Claude Code do
<code>/agentm2m:init [template]</code>	Check for an existing workspace (ask before replacing it), <code>team_init</code> , <code>team_validate</code> , report agents, views, hand-offs and the number of values waiting. For a custom team, hand the design to the <code>handoff-architect</code> subagent.
<code>/agentm2m:run [agent]</code>	run; while values are pending, dispatch one <code>binding-worker</code> per owning agent (in parallel), then run again; finally acceptance, reporting every escalation with a suggested source change.
<code>/agentm2m:change <change></code>	Turn the request into <code>model_edit</code> operations, preview them with <code>impact</code> and show the obligations, apply, run, and show with <code>trace_query</code> that untouched artefacts were not redone.
<code>/agentm2m:evolve <role></code>	Design a minimal view and hand-off for a new agent, <code>team_evolve</code> , run, report the obligations the new agent received.
<code>/agentm2m:status [key]</code>	Report the hand-written and/or <code>AUTO2M2M</code> team, open items per agent, φ , and the trace closure of an element.
<code>/agentm2m:author-handoff</code>	A reference for writing views and rules, with rules of thumb (keep footprints minimal; validators return reasons).
<code>/agentm2m:auto-build <file></code>	The <code>AUTO2M2M</code> loop: task, propose (via <code>team-builder</code>), admit (at most 3 rejected rounds), run with <code>auto-binding-workers</code> , φ , deliverable.
<code>/agentm2m:check [file]</code>	W1–W6 on a typed team; group diagnostics by condition; propose the smallest JSON fix for each.
<code>/agentm2m:diagnose</code>	φ failed: <code>auto_status</code> , <code>auto_attribute</code> , propose a minimal delta, submit it (checked), run again.
<code>agentm2m-concepts</code>	Not a command: background knowledge loaded automatically in projects with <code>.agentm2m/</code> .

Table 12. Subagents and their tool allowlists.

Subagent	Job	May call
<code>binding-worker</code>	Fill pending values of one agent of a hand-written team.	<code>next_bindings</code> , <code>submit_binding</code> . Nothing else.
<code>auto-binding-worker</code>	The same for an <code>AUTO2M2M</code> team.	<code>auto_next_bindings</code> , <code>auto_submit_binding</code> .
<code>team-builder</code>	Write a typed team for a task and iterate until admitted.	<code>Read</code> , <code>Grep</code> , <code>Glob</code> , <code>auto_status</code> , <code>auto_propose</code> , <code>auto_check</code> , <code>auto_submit_team</code> .
<code>handoff-architect</code>	Design views and rules for a custom team.	<code>Read</code> , <code>Grep</code> , <code>Glob</code> , <code>Edit</code> , <code>Write</code> , <code>team_status</code> , <code>team_validate</code> , <code>impact</code> , <code>model_show</code> , <code>auto_check</code> .

**Figure 7.** Context isolation. The worker starts with a fresh context; its allowlist stops it from fetching anything but its prompts. Only the dispatch sentence crosses the wall, and the skill fixes its wording.

The two designers get the opposite rights: they *should* read the project’s documentation to choose roles, and the architect may write rule files. But neither can fill values, and the team-builder cannot run the team: its instructions say “Never call run tools”, and its allowlist makes that true. Each subagent can do its job and little more.

Remark 5 (Parallel workers, one engine). `/agentm2m:run` dispatches one worker per agent *in parallel*. Inside one server process this is safe: each worker fetches and submits only its own agent’s values, the server runs one operation at a time, and a value submitted against an outdated footprint is refused as stale. A worker that finds only blocked values simply stops; `/agentm2m:run` calls `run` again and re-dispatches. Parallelism can cost a retry, but not a wrong stamp.

Q8. Check your understanding. A colleague proposes to add `Read` to binding-worker’s allowlist “so it can follow the project’s code style”. Explain, using the definition of a stamp, what would break.

9.3 The hooks

Session start. `session-start.sh` checks whether the project’s `.agentm2m/` directory contains `team.yaml`, `auto/team.json` or `auto/task.json`, and if so prints a short note that Claude Code adds to its context. It deliberately does *not* start the engine, so session start stays instant and works offline (Listing 16).

Listing 16. Observed: output of the `SessionStart` hook in a fresh `DevTeam` project.

```
agentm2m workspace detected at .agentm2m/ (team: devteam). Hand-offs between its agents are M2M
transformations run by the agentm2m MCP server: use /agentm2m:status to inspect, /agentm2m:run
to execute, /agentm2m:change for requirement changes. Never edit .agentm2m/state/ by hand.
```

After every edit. Claude Code passes the hook, on `stdin`, a JSON description of the tool call that just happened (Listing 17).

Listing 17. Illustrative: the input of a `PostToolUse` hook (abridged; the fields used here are real).

```
{"hook_event_name": "PostToolUse", "tool_name": "Edit",
 "tool_input": {"file_path": "/home/you/play/.agentm2m/rules/Req2Arch.agentm2m", ...}, ...}
```

The script extracts `tool_input.file_path` with a three-line inline Python program and then decides by the path:

- a file under `.agentm2m/state/`, or another file under `.agentm2m/auto/`: do nothing (engine-owned);
- `.agentm2m/auto/team.json` or any `*.typed-team.json`: run the `AUTO_M2M` checker (`agentm2m auto check`);
- a rule module, `team.yaml`, `helpers.py` or a view spec in `.agentm2m/`: run `agentm2m workspace validate -brief`;
- anything else: do nothing.

If validation fails, the script writes the errors to `stderr` and exits with code 2; Claude Code shows them to the LLM, which can fix the file at once. Listing 18 shows what it reports for a misspelt class name and for a typed team whose `Tester` lost its tools.

Listing 18. Observed: what the validation hook reports (we ran the commands the hook runs; file path shortened).

```
# a rule module with the typo to op : Arch!Operatoin (
agentm2m: workspace INVALID
- Req2Arch: unknown class or missing root slot 'Operatoin' (declare it in team.yaml)

# a *.typed-team.json in which the Tester has no tools
AutoM2M checker rejected bad.typed-team.json:
W6 Story2TestSuite.code needs ['exec']; owner Tester has []
REJECTED: 1 violation(s)
Fix the diagnostics (or submit through auto_submit_team).
```

Two robustness choices are worth copying. The hook runs the engine through `uvx` with the same pinned version as the MCP server. And if `uvx` is missing (or `python3`, which it needs to read its input), the hook exits silently with code 0: a missing tool must never block the user's editing. When we ran the hook on a machine without `uvx` in its `PATH`, this is exactly what happened; Listing 18 was produced by running the same commands directly.

Key takeaways.

- Skills that change a team preview or check first, and their reports are tied to tool results.
- Binding workers can call two tools only: they cannot fetch anything beyond their prompts. Designers can read and write files but cannot fill values or run the team.
- Hooks announce a team at start (offline) and re-check every edited team file, feeding errors back through exit code 2; they fail soft.

10 Walkthrough 1: a hand-written team in the plugin

We now follow a complete session with `DEVTEAM` (Section 3). Each step names the skill that triggers it; the listings show the tool outputs. To reproduce them without Claude Code, run `docs/plugin-walkthroughs/walkthrough1_devteam.py`.

What you see in the terminal (illustrative). You type `/agentm2m:init devteam`. Claude Code loads the `init` skill and asks your permission before calling the plugin's MCP tools (you can allow them once or for the session). It calls `team_status`, `team_init` and `team_validate`, and summarises: four agents, their views, three hand-offs, values waiting. You type `/agentm2m:run`. Claude Code calls `run`, then starts one binding-worker subagent per agent that has work; you see each subagent's progress as a collapsed line, and each returns a short report ("accepted 2, escalations: none"). Claude Code runs again until nothing is pending and ends with φ and any escalations. Below we show what happens *inside* these steps.

10.1 Step 1 (`/agentm2m:init`): create, validate, look

`team_init` copies the template into `.agentm2m/`; `team_validate` parses every rule module and counts matches (Listing 19).

Listing 19. Observed: `team_validate` on the fresh workspace (abridged).

```
{
  "ok": true,
  "errors": [],
  "handoffs": [
    {
      "name": "Req2Arch",
      "rules": [
        { "rule": "Epic2Component", "matches": 1, "stochastic_bindings": 0 },
        { "rule": "Story20peration", "matches": 2, "stochastic_bindings": 1 }
      ]
    },
    {
      "name": "Arch2Code",
      "rules": [
        { "rule": "Operation2CodeEdit", "matches": 0, "stochastic_bindings": 1 }
      ]
    },
    {
      "name": "Req2Test",
      "rules": [
        { "rule": "Criterion2TestCase", "matches": 3, "stochastic_bindings": 1 }
      ]
    }
  ]
}
```

`Story20peration` matches 2 stories (the guard excludes the draft); `Operation2CodeEdit` matches nothing yet, because no operation exists yet; `Criterion2TestCase` matches all 3 criteria. `team_status` gives the overview a student should look at first (Listing 20).

Listing 20. Observed: `team_status` before the first run (abridged).

```
{
  "team": "devteam",
  "backend": "host",
  "agents": {
    "Analyst": ["Req"],
    "Architect": ["Arch"],
    "Developer": ["Code"],
    "Tester": ["Test"]
  },
  "views": {
    "Req": { "owner": "Analyst", "elements": 7 },
    "Arch": { "owner": "Architect", "elements": 0 },
    ...
  },
  "handoffs": [
    {
      "name": "Req2Arch",
      "trace_links": 0,
      "bindings": { "uncovered": 2 },
      ...
    }
  ],
  "bindings": { "uncovered": 5, "phi": false, "open_total": 5 }
}
```

The five values are uncovered (Figure 6): the rules would match, but no target object exists yet.

10.2 Step 2 (/agentm2m:run): build the structure

Listing 21. Observed: the first run (abridged).

```
{
  "backend": "host",
  "passes": 2,
  "handoffs": {
    "Req2Arch": {
      "created": 3,
      "pending": 2,
      "blocked": 0
    },
    "Arch2Code": {
      "created": 2,
      "pending": 0,
      "blocked": 2
    },
    "Req2Test": {
      "created": 3,
      "pending": 3,
      "blocked": 0
    }
  },
  "pending": 5,
  "blocked": 2,
  "phi": false,
  "pending_preview": [
    {
      "target_key": "Story20operation::op::s=UserStory#S1",
      "binding": "signature",
      "agent": "Architect"
    },
    {
      "target_key": "Story20operation::op::s=UserStory#S2",
      "binding": "signature",
      "agent": "Architect"
    },
    {
      "target_key": "Criterion2TestCase::tc::c=Criterion#S1.1",
      "binding": "oracle",
      "agent": "Tester"
    },
    ...
  ]
}
```

Without any LLM call the engine created 8 objects (1 component, 2 operations, 2 code edits, 3 test cases) with their references and trace links. Within one pass the hand-offs run in the order of `team.yaml`, so `Arch2Code` already saw the operations that `Req2Arch` had just created; the second pass changed nothing, which tells the engine it has reached the fixpoint. Five values are pending. The two code bodies are *blocked*: their footprint contains a signature that has never been accepted.

10.3 Step 3 (inside a binding-worker): the prompts

`next_bindings` returns the ready values with their prompts, a remaining count, a `blocked_on_upstream` count, and an `instructions` field that repeats the rule “answer ONLY from its prompt”. Listing 22 renders two of the five values.

Listing 22. Observed: two of the five values returned by `next_bindings` (rendered).

```
target_key: Story20operation::op::s=UserStory#S1  binding: signature  agent: Architect
footprint_version: d550b926cae2fdb8              attempts: 0    kind: new    max_attempts: 3
prompt:
  Derive an API signature for this user story. Answer with exactly one line of the form
  name(param: Type, ...) -> ReturnType and nothing else.

  Context (footprint only):
  - Criterion({'id': 'S1.1', 'text': 'a title is required'})

target_key: Criterion2TestCase::tc::c=Criterion#S2.1  binding: oracle  agent: Tester
footprint_version: 123ccdc996aff09e
prompt:
  Write a test oracle for this acceptance criterion: Python code defining a function
  test_oracle() that calls implementation() and asserts the criterion holds. Answer with
  Python code only.

  Context (footprint only):
  completing a done task is rejected
```

The prompt is *everything* the writer of the value may know. Notice what is missing: the signature’s footprint is `s.criteria`, so the Architect does not even see the story’s title. That is a choice of this template, and it is visible to everyone who reads the rule. Whatever is not in the footprint cannot influence the value, and a later change to it will not make the value stale.

10.4 Step 4: a wrong answer

We first submit a sloppy signature, the way an LLM might (Listing 23).

Listing 23. Observed: submit_binding with the value “Here is the signature: create_task”.

```
{
  "target_key": "Story20operation::op::s=UserStory#S1",
  "binding": "signature",
  "handoff": "Req2Arch",
  "status": "rejected",
  "reason": "expected exactly one line: name(param: Type, ...) -> ReturnType",
  "attempts": 1,
  "attempts_left": 2,
  "retry_prompt": "Derive an API signature for this user story. ... Context (footprint only):\n- Criterion({'id': 'S1.1', 'text': 'a title is required'})\nYour previous answer was rejected (expected exactly one line: name(param: Type, ...) -> ReturnType):\nHere is the signature: create_task\nAnswer again, following the requested output format exactly."
}
```

The reason comes from `parses()` in Listing 4. Because the validator explains itself, the retry can be precise.

10.5 Step 5: fill everything; φ holds

We then submitted acceptable values, in this order:

1. the signatures `create_task(title: str) -> Task` and `mark_done(task_id: int) -> Task: accepted`;
2. three oracles, for example for S1.1:

Listing 24. Illustrative: the oracle we submitted for S1.1 (accepted).

```
def test_oracle():
    try:
        implementation('')
    except ValueError:
        return
    raise AssertionError('empty title accepted')
```

all accepted by `failsOnStub()`;

3. a new `next_bindings`: the signatures being accepted, the two code bodies are now unblocked and offered, with the signature and the copied criteria in their prompts;
4. two code bodies, both the deliberately useless `def f(x): return x`: both accepted.

The next run reported `"pending": 0, "phi": true`; all seven values are fresh.

Remark 6 (A weakness made visible). Both code bodies were accepted because the template checks them only with `compiles()`, a *form* validator (red in Figure 2). φ holds, yet nothing checked that the code does what S2.1 asks. If this team were written as a typed team, the AUTOM2M checker would reject it under W4 (“view Code reaches no behavioural check”); the companion reports the same finding for the hand-written pilot team, together with Arch [1]. φ is only as meaningful as the validators it counts.

Q9. Check your understanding. Which validator let `def f(x): return x` through, and which admission condition would reject this team design?

10.6 Step 6 (/agentm2m:change): a change, previewed before it costs anything

The Analyst tightens S2.1 to “completing a done task returns HTTP 409”. The /agentm2m:change skill first looks at the source (model_show), turns the request into an edit operation, and previews it with impact, which applies the edit to a scratch copy, runs the structural phase and calls no LLM (Listing 25).

Listing 25. Observed: impact for the edit {"op": "set", "key": "Criterion#S2.1", "values": {"text": ...}} of view Req as the Analyst.

```
{
  "obligations": [
    {
      "handoff": "Req2Arch",
      "target_key": "Story20operation::op::s=UserStory#S2",
      "binding": "signature",
      "agent": "Architect",
    },
    {
      "handoff": "Arch2Code",
      "target_key": "Operation2CodeEdit::ce::op=Operation#op_s2",
      "binding": "body",
      "agent": "Developer",
    },
    {
      "handoff": "Req2Test",
      "target_key": "Criterion2TestCase::tc::c=Criterion#S2.1",
      "binding": "oracle",
      "agent": "Tester",
    }
  ],
  "new_bindings": [],
  "created": [],
  "deleted": [],
  "llm_calls_upper_bound": 9,
  "note": "direct obligations only: a re-sampled value that changes may oblige downstream bindings on the next run"
}
```

Exactly three values must be redone: S2’s signature (its footprint is S2’s criteria), S2’s code body (its footprint contains the criteria copied onto the operation) and S2.1’s oracle. Nothing of S1 is touched. The upper bound of 9 LLM calls is 3 obligations times $k = 3$. The note matters: if the Architect’s new signature differs from the old one, the code body’s footprint changes *again*, one hop further; if it is identical, propagation stops. (The plugin’s tests check both cases.) Figure 8 shows why the engine finds exactly these three.

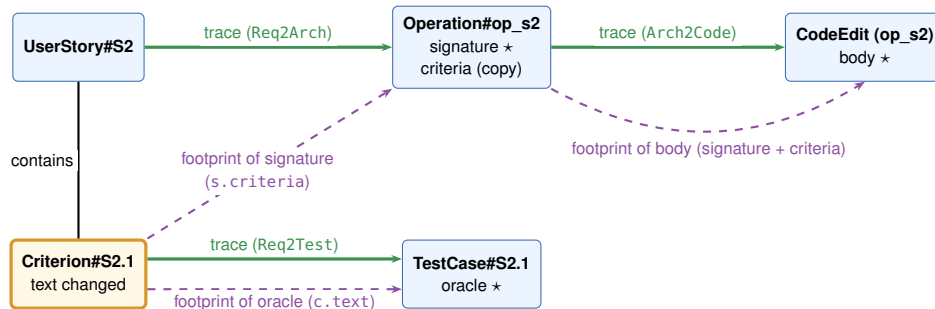


Figure 8. Trace links versus footprints for the change of S2.1. Solid green arrows are trace links (“created from”): trace_query follows them. Dashed purple arrows are footprint reads (“written from”): impact compares their stamps. The three starred values are the obligations.

Before applying the edit, two mistakes the engine refuses (Listing 26), and the reason for the second (Listing 27).

Listing 26. Observed: model_edit refusals.

```
model_edit(view="Req", ..., as_agent="Architect")
write rights (omega): agent Architect may not write view Req; its owner is Analyst
model_edit(view="Arch", ops=[set Operation#op_s2 name], as_agent="Architect")
cannot modify Operation#op_s2: it is engine-owned (created by hand-off rule Story20operation);
change its source view instead
```

Listing 27. Observed: model_show(view="Arch", key="Operation#op_s2").

```
{
  "view": "Arch",
  "element": {
    "type": "Operation",
    "key": "Operation#op_s2",
    "engine_owned": "Story20operation::op::s=UserStory#S2",
    "name": "op_s2",
    "signature": "mark_done(task_id: int) -> Task",
    "criteria": "S2.1: completing a done task is rejected",
    "component": "Arch:Component#E1"
  }
}
```

The Architect owns view Arch, but op_s2 was *created by a rule*: it is *engine-owned*, as model_show says. If an agent could edit it, the next run would either overwrite the edit or contradict the trace. To change it, change its source.

Applied as the Analyst, the edit reports "open_obligations": 3, and next_bindings offers the three values with "kind": "stale" and new prompts. The Developer's prompt shows the *old* signature next to the *new* criterion (Listing 28). It is not blocked, because the old signature was once accepted (Section 8.6). If the Developer answered now and the Architect's new signature then differed, the Developer's answer would be refused as stale or become an obligation again; either way no wrong stamp results.

Listing 28. Observed: the Developer's stale prompt after the change (rendered).

```
target_key: Operation2CodeEdit::ce::op=0operation#op_s2  binding: body  kind: stale
prompt:
  Implement this operation as one self-contained Python function: first the signature, then
  the acceptance criteria it must satisfy. Answer with Python code only.

Context (footprint only):
- mark_done(task_id: int) -> Task
- S2.1: completing a done task returns HTTP 409
```

10.7 Step 7 (/agentm2m:status): what came from what

trace_query answers "what was derived from this element?". For Criterion#S2.1 it returns one element; for the story, two (Listing 29).

Listing 29. Observed: trace_query for the criterion and for its story (abridged).

```
trace_query("Criterion#S2.1")
  downstream: [{"from": "Criterion#S2.1", "handoff": "Req2Test", "target": "Test:TestCase#S2.1"}]
trace_query("UserStory#S2")
  downstream: [{"from": "UserStory#S2", "handoff": "Req2Arch", "target": "Arch:Operation#op_s2"},
                {"from": "Operation#op_s2", "handoff": "Arch2Code", "target": "Code:CodeEdit#op_s2"}]
```

Q10. Check your understanding. impact found three obligations for a change of Criterion#S2.1, but trace_query on the criterion lists only one derived element. Using Figure 8, explain why both answers are right.

10.8 Step 8 (/agentm2m:evolve): grow the team while it runs

/agentm2m:evolve adds a Security Reviewer with the ready-made view Sec and hand-off Arch2Sec from the template's rules/extra/. This is a *higher-order transformation* [15]: a change of the team itself, while it runs. team_evolve reported "existing_matches": 2; the next run created two SecurityReview objects and offered four new values (notes and risk for each operation; Listing 15 is one of them) next to the three stale ones from Step 6. The new agent received work for everything that already existed, without glue code. The evolution is stored in state.json, so it survives restarts, and from now on trace_query on UserStory#S2 also lists the review of op_s2.

10.9 Step 9: back to φ

We filled the seven pending values: the same signature for op_s2 (so the code body did not become stale a second time), a new oracle and code body for the 409 behaviour, and two notes and two risk ratings. All were accepted; the final run reported "phi": true and team_status "bindings": {"fresh": 11} with one evolution (SecurityReviewer).

Key takeaways.

- One run builds all structure for free; values are filled one prompt at a time, and blocked values wait for their inputs.
- Validators that return reasons turn a rejection into a precise retry.
- `impact` previews a change exactly, before any token is spent: it follows footprints, while `trace_query` follows trace links.
- Write rights and engine ownership stop edits in the wrong place.
- φ is only as strong as the validators: a compile-only check lets wrong code through.

11 Walkthrough 2: AUTOM2M in the plugin

In the first walkthrough a person had designed the team. Now Claude Code designs it, as the *builder*, and the checker decides whether it may run. In Claude Code you type `/agentm2m:auto-build wallet.py`. Figure 9 shows the loop the skill drives. The task is illustrative; all engine outputs are observed (script `walkthrough2_autom2m.py`).

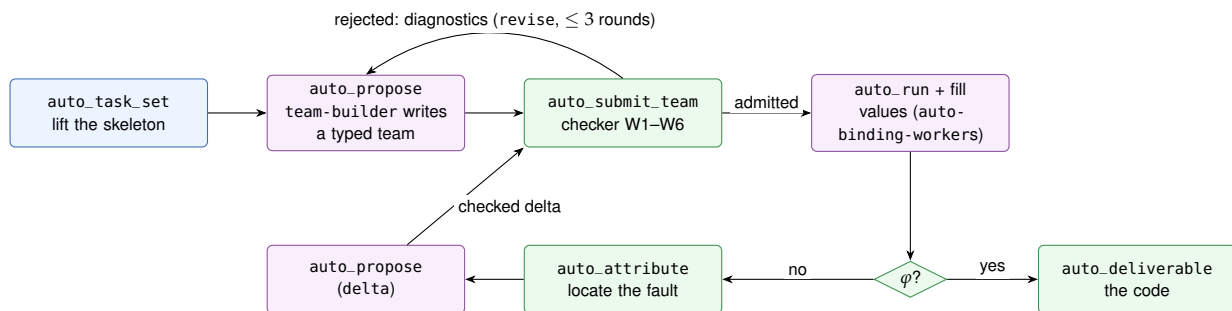


Figure 9. The AUTOM2M loop as `/agentm2m:auto-build` and `/agentm2m:diagnose` drive it. Purple steps are written by Claude Code or its subagents; green steps are decided by the checker or the engine.

11.1 The task

AUTOM2M currently handles Python tasks: a class whose methods are *stubs*, or one stub function. A stub has a docstring and a body of `pass`, `...` or `raise NotImplementedError`. Docstring lines that start with `>>>` are *doctest examples*: a call and, on the next line, its expected output.

Listing 30. Illustrative: the task `wallet.py`.

```
class Wallet:
    """A wallet that holds money in cents."""

    def __init__(self):
        self.cents = 0

    def deposit(self, cents):
        """
        Add money to the wallet.
        :param cents: int, must be positive
        :return: int, the new balance
        >>> w = Wallet()
        >>> w.deposit(250)
        250
        """
        pass

    def withdraw(self, cents):
        """
        Remove money; raise ValueError if the balance would go below zero.
        :return: int, the new balance
        >>> w = Wallet(); _ = w.deposit(300)
        >>> w.withdraw(120)
        180
        """
        pass
```

`auto_task_set` parses the source and *lifts* it into the fixed Goal view: one Task object (name, description, skeleton) and one Method object per stub (name, signature, docstring, examples). It found `deposit` and `withdraw`, both with examples; `__init__` is already implemented and stays part of the frame into which the methods will later be assembled.

11.2 Step A: the builder prompt

`auto_propose` chose mode `propose` (there is no team yet) and returned a prompt of 8,587 characters. It contains, in order: the JSON format of a typed team; the fixed Goal view; the *validator library*, the only validators a team may use (Listing 31); the six admission conditions in plain words; a worked example (a three-agent team for a parking meter); and the task.

Listing 31. Observed: the validator library as the builder prompt lists it.

```
- nonempty() [form]: value is non-empty text
- json() [form]: value is a JSON document
- compiles() [form]: value contains Python code that parses
- defines(method: a Goal object) [form]: value defines a function with the method's name
- runs() [behaviour] needs tools ['exec']: value is Python code that executes without raising
- examples_run(method: a Goal object) [behaviour] needs tools ['exec']: the method, spliced
  into the class, runs the method's public docstring examples without raising
- examples_match(method: a Goal object) [behaviour] needs tools ['exec']: as examples_run,
  and every printed result equals the documented one
- passes_tests(method: a Goal object, tests: string) [behaviour] needs tools ['exec']: the
  method, spliced into the class, passes the given test code
- test_valid(method: a Goal object) [behaviour] needs tools ['exec']: value is test code that
  calls the method and fails on a stub implementation
```

Each validator declares its *strength* (form or behaviour) and the tools its owner needs. These declarations make W4 (“every goal reaches a behavioural check”) and W6 (“the owner has the tools”) decidable from the JSON alone. Only behaviour validators execute code.

11.3 Steps B and C: a rejected proposal, then an admitted one

Our first proposal had the shape of the worked example: a Designer who plans each method, a Tester who writes tests, a Developer who implements. We made four deliberate mistakes (Listing 32).

Listing 32. Illustrative: the four faulty fragments of the first proposal.

```
"agents": [..., {"name": "Tester", "role": "...", "tools": []}, ...]      (no exec)
"writes": {"Designer": ["Design"], "Tester": ["Test"], "Developer": ["Code", "Test"]}
... "footprint": ["d.method.task.skeleton", ..., "t.code", "d.method.returnType"] ...
"done": ["cover(G)", "valid", "fresh", "noObl", "Tester.says('ALL TESTS PASS')"]
```

Listing 33. Observed: auto_submit_team on the first proposal.

```
W1 Method2Impl.code: footprint path 'd.method.returnType': Goal!Method has no feature
  'returnType' (its features are ['docstring', 'examples', 'name', 'signature', 'task'])
W2 view Test written by ['Developer', 'Tester']; needs exactly one writer
W5 done-clause Tester.says('ALL TESTS PASS') is not engine-checkable
W6 Method2Test.code needs ['exec']; owner Tester has []
REJECTED: 4 violation(s)
next: fix every diagnostic and resubmit (auto_propose gives a revise prompt)
```

Each diagnostic names the condition, the exact element and what is wrong; the W1 message even lists the features that *do* exist. The rejected proposal is saved in auto/proposal.json, and the next auto_propose switches to mode revise: the same format and rules, the rejected JSON, and “Checker diagnostics (each must be fixed)”. /agentm2m:auto-build allows three rejected rounds before it stops and shows the diagnostics to you.

Our corrected team was admitted in round 2, with "mode": "admitted", agents Designer, Tester, Developer and hand-offs Goal2Design, Goal2Test, Build2Code. In fairness: apart from its name, it is exactly the worked example from the builder prompt. That is not a design achievement; it shows how strongly builders copy the example they are given, which the project’s preliminary experiments also report (100 of 103 final teams had the example’s shape; README.md). Appendix C prints the whole team; Listing 34 shows its most interesting rule.

Listing 34. Code excerpt: the rule that writes the deliverable (from Appendix C).

```
{
  "name": "Build2Code",
  "sources": ["Goal", "Design", "Test"],
  "target": "Code",
  "rules": [
    {
      "name": "Method2Impl",
      "from": [
        { "var": "d", "type": "Design!MethodDesign" },
        { "var": "t", "type": "Test!MethodTest" }
      ],
      "guard": "t.method = d.method",
      "to": { "var": "i", "type": "Code!MethodImpl" },
      "bind": { "name": "d.method.name", "method": "d.method" },
      "llm": [
        {
          "feature": "code",
          "prompt": "Implement this method of the class.",
          "footprint": [
            "d.method.task.skeleton",
            "d.method.signature",
            "d.method.docstring",
            "d.plan",
            "t.code"
          ],
          "validator": [
            {
              "id": "examples_run",
              "args": { "method": "d.method" }
            },
            {
              "id": "passes_tests",
              "args": { "method": "d.method", "tests": "t.code" }
            }
          ]
        }
      ]
    }
  ]
}
```

Read it like a hand-off rule: for every pair of a design and a test of the same method, create one implementation; its name and method are copied by the engine; its code is written by an LLM that sees the skeleton, the signature, the docstring, the plan and the tests, and is accepted only if the documented examples run *and* the Tester’s tests pass.

11.4 Step D: run and fill

The first `auto_run` created the structure and reported 4 values pending (two plans, two test suites) and 2 *blocked* (the two implementations, whose footprints contain a plan and a test that have never been accepted). φ was false with `cover(G)` and `fresh open`, which simply means “nothing is done yet”.

AUTOM2M prompts are richer than hand-written ones: the agent’s role, then the task, then a precise output format, then the footprint, each part labelled by its path (Listing 35).

Listing 35. Observed: the Developer’s prompt for `withdraw` (skeleton and tests abridged).

```
You are the Developer. You implement one method following its design so that its tests and
examples pass.

Task: Implement this method of the class.

Answer with the complete Python definition of the method named in the context (signature line
and body) in ONE ```python code block. Do not repeat the class.

Context (footprint only):
- [d.method.task.skeleton]
class Wallet: ...
- [d.method.signature]
def withdraw(self, cents):
- [d.method.docstring]
Remove money; raise ValueError if the balance would go below zero. ...
- [d.plan]
- if cents > self.cents raise ValueError
- subtract and return the new balance
- [t.code]
```python
import unittest
class TestWithdraw(unittest.TestCase):
 def test_subtracts(self): ...
 def test_overdraw_rejected(self):
 w = Wallet(); w.deposit(100)
 with self.assertRaises(ValueError):
 w.withdraw(1000)
...

```

We submitted the plans (validator `nonempty`) and the test suites (validator `test_valid`: the tests must call the method and *fail* on a stub). Tests must be `unittest.TestCase` classes (as the Tester’s prompt asks) or functions named `test_*`; in an earlier attempt we wrote plain `assert` lines at module level, which run while the file loads, against the stub, and were rejected (“test code does not load”). All four were accepted. The next `auto_run` unblocked the two implementations. The `deposit` implementation was accepted. For `withdraw` we submitted, three times, code that forgets the overdraft check:

**Listing 36.** Illustrative: the wrong `withdraw` we submitted.

```
def withdraw(self, cents):
 self.cents -= cents
 return self.cents

```

**Listing 37.** Observed: a behaviour validator rejects it (first of three attempts; the third returned escalated).

```
{
 "target_key": "Method2Impl::i::d=MethodDesign#...#withdraw|t=MethodTest#...#withdraw",
 "binding": "code", "handoff": "Build2Code", "status": "rejected",
 "reason": "failing tests: AssertionError: ValueError not raised",
 "attempts": 1, "attempts_left": 2, "retry_prompt": "You are the Developer. ..."}

```

Compare this with Step 5 of the first walkthrough: there, a compile-only validator let wrong code through. Here the code is *executed* against the documented examples and the Tester’s tests, in the sandbox (Section 13), and the reason names the failing assertion.

## 11.5 Step E (/agentm2m:diagnose): $\varphi$ fails; locate the fault

With nothing pending, `auto_run` reported  $\varphi$  false and `"deliverables_accepted": {"deposit": true, "withdraw": false}`. `auto_attribute` turned each failed clause into a located fault (Listing 38).

**Listing 38.** Observed: `auto_attribute` in host mode (abridged).

```
{
 "phi": false,
 "faults": [
 {
 "clause": "noobl",
 "handoff": "Build2Code",
 "rule": "Method2Impl",
 "binding": "code",
 "agent": "Developer",
 "fault_class": "unclassified",
 "summary": "noobl at Build2Code/Method2Impl.code (agent Developer): failing tests:
 AssertionError: ValueError not raised",
 {
 "clause": "fresh",
 "...",
 "reason": "a stamp does not match its current footprint"
 },
 {
 "clause": "cover(G)",
 "goal_element": "Method#withdraw",
 "reason": "Method#withdraw has no accepted deliverable"
 }
],
 "note": "host mode: located only; classification replays need an engine LLM"
]
}
```

The location is exact and costs no LLM call: hand-off `Build2Code`, rule `Method2Impl`, value `code`, agent `Developer`, with the validator’s reason. The three faults are one problem seen from three clauses: an escalated value (`noobl`), a value without a valid stamp (`fresh`), and a method without a deliverable (`cover(G)`).

What host mode cannot do is *classify* the fault. With an engine LLM, `auto_attribute` replays the failing value in a fixed order and reports the first class that fits (Table 13). Replays need an LLM the engine can call by itself; in host mode the LLM is Claude Code, so faults stay unclassified and `/agentm2m:diagnose` tells Claude Code to decide from the reason.

**Table 13.** Fault classes of AUTOM2M attribution (`auto/attribution.py`), checked in this order.

Class	What the replay shows	Remedy
validator	the failing validator gives different verdicts on the same value	fix the validator (it is not deterministic)
sampling	a replay on the <i>same</i> footprint passes	retry, or a stronger model
footprint	a replay on a footprint widened by one hop passes	widen the footprint in the team
upstream	a footprint value was written by an LLM upstream, and redoing it lets this value pass	redo the upstream value
specification	none of the above	change the prompt or the validator
coverage	(a <code>cover(G)</code> failure with no single value to blame)	change the team’s shape

## 11.6 Step F: a checked repair

Here the Developer had everything it needed in its footprint: `docstring`, `plan` and `test` all say “raise `ValueError`”. This looks like a *sampling* fault, whose remedy is simply another try. But the value is escalated, and an escalated value is not offered again on an unchanged footprint. `/agentm2m:diagnose` therefore asks for a *delta*: `auto_propose` in mode `delta` returns the current team and the fault summaries and asks for “a minimal change to the team that removes them”. Our delta added the `doctest` examples (`d.method.examples`) to the footprint of `Method2Impl.code`.

**Listing 39.** Observed: `auto_submit_team` with the delta.

```
{
 "admitted": true,
 "report": "ADMITTED",
 "mode": "in-place",
 "kept_values": 5,
 "agents": ["Designer", "Tester", "Developer"],
 "handoffs": ["Goal2Design", "Goal2Test", "Build2Code"]
}
```

The delta passed the same checker before anything changed. How it is applied depends on what it changes (Table 14). Ours only edited a rule of an existing hand-off, so it was applied *in place*.

**Table 14.** How an admitted delta is applied to the running team (auto/repair.py).

Mode	When	What happens to accepted values
noop	the team is unchanged	nothing
in-place	only rules of existing hand-offs change (prompts, footprints, validators, rules)	kept; values whose prompt or validator changed lose their stamp; values whose footprint changed become stale; <i>all escalations are cleared</i>
hot	views, agents or hand-offs are only added	kept; the additions get fresh trace models; all escalations are cleared
rebuild	views, classes or write rights change, or a hand-off is removed or retargeted	discarded: the team starts from scratch

"kept\_values": 5 counts the stamps still stored after the delta: two plans, two test suites and the deposit code. Only values written by the changed rule have a new footprint, so the plans and tests were not redone; but both implementations, including the accepted deposit code, came back as pending, because their footprint now also contains the examples. This time both implementations were accepted, the next auto\_run reported "phi": true, and auto\_deliverable assembled the class (Listing 40).

**Listing 40.** Observed: auto\_deliverable with "complete": true (assembly by the engine; method bodies typed by us).

```
class Wallet:
 """A wallet that holds money in cents."""

 def __init__(self):
 self.cents = 0

 def deposit(self, cents):
 self.cents += cents
 return self.cents

 def withdraw(self, cents):
 if cents > self.cents:
 raise ValueError('insufficient funds')
 self.cents -= cents
 return self.cents
```

auto/history.json recorded the whole story: the task; admission round 1 rejected (W1, W2, W5, W6); round 2 admitted; two runs; one attribution; the delta as round 3, admitted in-place with 5 values kept; the final run.

**Remark 7 (Be honest about what the repair did).** Widening the footprint added no information the Developer lacked. And it was not even what re-opened withdraw: every admitted in-place or hot delta clears *all* escalations (Table 14), so a delta that only reworded a prompt would have done the same. In this example the correct answer came from us. With an engine LLM, auto\_attribute would have replayed the value on its unchanged footprint and, if a replay passed, reported a sampling fault, for which the right remedy is a retry or a stronger model rather than a team change. In host mode that judgement is left to Claude Code and to you.

**Q11. Check your understanding.** (a) Why were the two implementations blocked after the first auto\_run? (b) After the delta, why were the plans and tests not redone, while the accepted deposit code was? (c) Name a delta that changes no footprint at all but would still have re-opened withdraw.

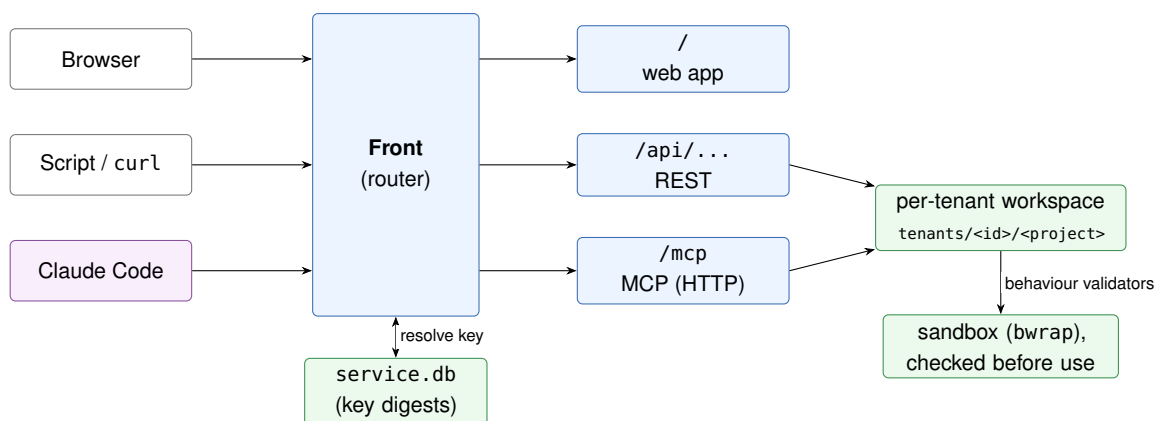
**Key takeaways.**

- In the plugin, Claude Code is the AUTOM2M builder; the checker returns exact diagnostics and the revise prompt carries them back.
- Behaviour validators execute the code against examples and tests, so a wrong implementation is rejected with the failing assertion.
- In host mode faults are located exactly but not classified; repairs are deltas that pass the same checker, keep accepted values where they can, and clear escalations.

## 12 The hosted service

The plugin runs the engine on the developer's machine. Version 0.3.0 adds a second way: one server, started with `agentm2m serve`, offers the same tools to many users over the web (requirement R6).

### 12.1 One process, three faces



**Figure 10.** The hosted service. One process serves the web site, the REST API and the MCP endpoint; REST and MCP reach the same per-tenant workspaces, and all code execution goes through the sandbox. (The web app's Playground calls the REST endpoint `/api/auto/check`.)

Figure 10 shows the structure (`src/agentm2m/server/app.py`). `/` serves the project's web site, including a *Services* tab with five pages: Status, API keys, a checker Playground (paste a typed team, see its W1–W6 diagnostics), API docs and Pricing. `/api/...` is a REST API whose documentation is generated at `/api/docs`. `/mcp` is the MCP endpoint, mounted from the *same* server object as the stdio server, so it offers the same 24 tools. A small router in front, *Front*, sends each request to the right part. It is an ASGI application: ASGI is Python's standard interface between a web server and a web application [16], so one ASGI application can forward to others. Starting the service is one command, or a container:

**Listing 41.** Code excerpt: starting the hosted service (from README.md).

```

pip install -e ".[serve]" && (cd web && npm ci && npm run build)
agentm2m serve --port 8765 # http://localhost:8765, MCP at /mcp, API docs at /api/docs
or
docker compose up --build

```

The health endpoint tells an operator what the service can do. We started the service in-process with a throw-away data directory (script `rest_demo.py`) on a machine without bubblewrap (Listing 42).

**Listing 42.** Observed: GET /api/health (abridged).

```
{
 "status": "ok",
 "version": "0.3.0",
 "mcp": {
 "endpoint": "/mcp",
 "transport": "streamable-http",
 "auth": "Authorization: Bearer <api key>",
 "tools": ["team_init", "team_status", ...],
 "tool_count": 24
 },
 "llm": {
 "mode": "host",
 "solve_backend": null
 },
 "execution": {
 "sandbox": "process",
 "isolated": false,
 "enabled": false,
 "reason": "code execution is disabled: this server has no isolating sandbox (set AGENTM2M_SANDBOX=bwrap, or AGENTM2M_SANDBOX=command with AGENTM2M_SANDBOX_CMD)"
 },
 "keys": {...},
 "metrics": {...},
 ...
}
```

## 12.2 API keys

Anyone can create a key with POST /api/keys (or on the web page), giving an optional label and an optional, unverified e-mail address; there is no login. The key is `am2m_` followed by 43 random URL-safe characters. It is returned *once*; the server stores only its SHA-256 digest, its first characters (so you can recognise it in a list), the label, a tier and usage counters, in SQLite. A key is presented as `Authorization: Bearer am2m_...` (or `X-API-Key`) and revoked with DELETE /api/keys/me. Every key is on the free tier; the function that would restrict features by tier, enforce, exists but deliberately does nothing yet.

## 12.3 Tenants and projects

Each key is a *tenant*: a separate customer whose data must never mix with another's. A tenant can have several *projects*, chosen with the header `X-AgentM2M-Project` (default default). Each project is an ordinary project directory on the server, `$AGENTM2M_DATA_DIR/tenants/<key id>/<project>/`, holding a `.agentm2m/` workspace exactly like a local one. Project names must match `[A-Za-z0-9_.-]{1,64}` and may not be `.` or `..`, so a name cannot climb out of the tenant's directory.

## 12.4 A REST session

Listing 43 drives the Wallet task of Section 11 through the REST API. Look at the last three answers: they are the service protecting itself.

**Listing 43.** Observed: a REST session against the service (abridged).

```
POST /api/keys {"label": "student"} -> 201 {"key": "am2m-EKwI...Upu0",
 ..., "record": {"id": "084e1ffa7a89d459", "tier": "free",
 "note": "Store the key now: it is shown only once ..."}}
POST /api/auto/task (no key) -> 401 {"detail": "missing or invalid API key"}
POST /api/auto/task {"source": "<wallet.py>"} -> 200 {"task": {"entry": "Wallet", "methods": [...]}}
GET /api/auto/propose -> 200 {"mode": "propose", "prompt": "<8587 chars>"}
POST /api/auto/team {"team": {...}} -> 200 {"admitted": true, "mode": "admitted", ...}
POST /api/auto/run -> 200 {"phi": false, "pending": 4, "blocked_on_upstream": 2}

POST /api/auto/bindings {<a test suite>
 ...}
POST /api/auto/solve -> 501 {"detail": "no engine LLM configured on this server
 (AGENTM2M_SOLVE_LLM); use host mode"}
POST /mcp (no key) -> 401 {"error": "unauthorized", "detail": "send an API key
 ..."}
```

Everything that needs no code execution worked: creating a key, setting the task, the builder prompt, the checker, the run. The first submission whose validator would have *executed* code was refused with 403, because this server had no isolating sandbox (Section 13). The directory `tenants/084e1ffa7a89d459/` now holds the project.

Errors map to HTTP status codes: 401 missing or invalid key; 400 invalid project name; 422 a malformed request body; 409 a workspace error (“no task yet”); 403 code execution refused by the sandbox policy; 501 `auto_solve` without a server LLM; 500 anything unexpected. Table 15 lists the endpoints. The REST API covers the AutoM2M tools; the hand-written-team tools are available over MCP only.

**Table 15.** REST endpoints of the hosted service.

Method and path	Key?	Purpose / equivalent tool
GET /api/health	no	status, tool list, sandbox status, counters
GET /api/pricing	no	the tiers
POST /api/keys	no	create a key (shown once)
GET, DELETE /api/keys/me	yes	show or revoke the presented key
POST /api/auto/check	no	<code>auto_check</code> (stateless; used by the playground)
GET /api/teams/samples	no	sample typed teams
POST /api/auto/task	yes	<code>auto_task_set</code>
GET /api/auto/propose	yes	<code>auto_propose</code>
POST /api/auto/team	yes	<code>auto_submit_team</code>
POST /api/auto/run	yes	<code>auto_run</code>
GET, POST /api/auto/bindings	yes	<code>auto_next_bindings</code> , <code>auto_submit_binding</code>
GET /api/auto/status	yes	<code>auto_status</code>
POST /api/auto/attribute	yes	<code>auto_attribute</code>
GET /api/auto/deliverable	yes	<code>auto_deliverable</code>
POST /api/auto/solve	yes	<code>auto_solve</code>
DELETE /api/auto	yes	<code>auto_reset</code>

## 12.5 How an MCP call learns who is calling

The MCP tools are the same functions as in the stdio server. How does a tool know which tenant called? Listing 44 shows the answer: the router authenticates the key and adds a *trusted* header naming the tenant; the tool reads it through `ctx.headers`.

**Listing 44.** Code excerpt: `Front.__call__` (server/app.py, abridged).

```
never trust a client-supplied tenant header
headers = [(k, v) for k, v in scope.get("headers", []) if k.lower() != b"x-agentm2m-tenant"]
...
rec = await asyncio.to_thread(self.keys.resolve, key_from_headers(hmap))
if rec is None: # -> 401 {"error": "unauthorized", ...}, WWW-Authenticate: Bearer
 ...
headers.append((b"x-agentm2m-tenant", rec.id.encode()))
await self.mcp_app(scope, receive, send)
```

The first line is the security-critical one. If the router did not remove a tenant header sent by the *client*, anybody with any valid key could name another tenant’s id and work in that tenant’s projects. Rule of thumb: a header that carries authority must be set by your own trusted code, and anything a client sends under that name must be thrown away.

**Q12. Check your understanding.** Describe the attack that would succeed without the first line of Listing 44. What would the attacker need?

## 12.6 Two ways to connect Claude Code

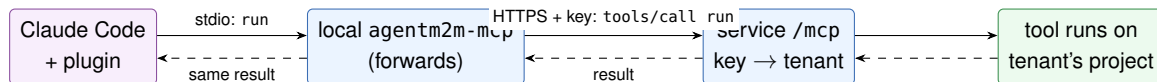
**Directly, without the plugin.** Register the endpoint as an MCP server:

**Listing 45.** Code excerpt: connecting Claude Code directly (from the plugin’s README.md).

```
claude mcp add --transport http agentm2m https://<service-url>/mcp --header "Authorization: Bearer <api key>"
```

Nothing is installed locally, and the tools appear as `mcp__agentm2m__*`. You get the tools, but not the skills, subagents and hooks, whose tool names refer to the plugin’s server.

**Through the plugin.** Start Claude Code with `AGENTM2M_URL=https://<service-url>` and `AGENTM2M_API_KEY=<key>` (and optionally `AGENTM2M_PROJECT`). The plugin still starts its local `agentm2m-mcp` through `uvx`, but that server now *forwards every tool call* to the service under the same tool name (Figure 11). The tool-calling skills and subagents work unchanged, and the team’s state lives on the service. Everything that edits or checks *local files* does not reach the service: the hooks validate local files and announce only a local workspace; `handoff-architect` and `/agentm2m:author-handoff` write rule files on your disk, which the service never sees. On the service, a hand-written team therefore starts from a template (`team_init`) and grows through `team_evolve`, whose `rule_text` creates rule files remotely.



**Figure 11.** The plugin as a proxy for the hosted service.

This is the *Proxy* design pattern [17]: an object with the same interface as the real one, which forwards each call. One decorator on every tool implements it (Listing 46).

**Listing 46.** Code excerpt: `_forwardable` (`mcp_server.py`, abridged).

```
def _forwardable(fn):
 """Run the tool here, or, when the plugin is pointed at a hosted service
 (AGENTM2M_URL + AGENTM2M_API_KEY), forward the same call there. Calls
 arriving over HTTP (i.e. on the hosted service itself) always run here."""
 def wrapper(*args, **kwargs):
 over_http = ... # does the request carry HTTP headers?
 rc = None if over_http else _remote_client()
 if rc is None:
 return fn(*args, **kwargs) # local mode
 arguments = {k: v for k, v in bound.arguments.items() if k != "ctx"}
 return rc.call_tool(fn.__name__, arguments) # same name, same arguments
 return wrapper
```

The same module runs on the server, which is why calls that arrive over HTTP always run locally (the guard in the docstring). The forwarding client (`src/agentm2m/remote.py`) is a minimal MCP client over Streamable HTTP: it initialises once, remembers the session id, re-initialises if the server answers 404 (it has forgotten the session, for example after a restart), and turns a 401 into the actionable message “create a new one on its web page (Services → API keys)”.

**Q13. Check your understanding.** What would happen without the guard “calls arriving over HTTP always run here” if the server’s own environment contained `AGENTM2M_URL` pointing at itself?

**Remark 8 (Three hosts).** “Host” means three different things in this document. The *MCP host* is the application that owns the LLM (Claude Code). *Host mode* means that host writes the values. The *hosted service* is the web server. On the hosted service, host mode still means that *your* Claude Code, the client, writes the values: by default the service never calls an LLM. An operator can set `AGENTM2M_SOLVE_LLM` (for example `anthropic`, with an API key) to enable `auto_solve`, the unattended loop with

at most 3 admission rounds and 2 repairs. `auto_solve` also works locally when `AGENTM2M_LLM` names an engine LLM.

#### Key takeaways.

- `agentm2m serve`: web app, REST API and MCP endpoint in one process, over the same service layer.
- API keys are shown once and stored as digests; each key is a tenant with its own project directories.
- Authority (the tenant) comes from a header only the server sets; a spoofed one is stripped.
- The plugin can act as a proxy: the tools and skills run against the service, but anything that works on local files stays local.

## 13 Security and trust

The plugin and the service execute code they did not write. This section collects what can go wrong and what defends against it (requirement R7).

### 13.1 Two kinds of foreign code

**Project helpers.** A rule module may say uses `'helpers.py'`; the engine then imports that Python file to get its helpers and validators. Importing a file *runs* it. Opening a workspace is therefore like running `make` in a repository: only do it for projects you trust. The engine refuses paths that leave `.agentm2m/` (a uses clause or rule path such as `../x.py`), and `team_evolve`'s `rule_text` can only create `.agentm2m` rule files, never Python files: otherwise an LLM could write a helper file and a rule that imports it. On the hosted service this matters most: a tenant can create workspaces only from the built-in templates and extend them with rule files, so a tenant cannot place its own `helpers.py` on the server.

**LLM-written values.** Behaviour validators execute code that an LLM wrote: the `AUTO_M2M` library validators (`examples_run`, `passes_tests`, `test_valid`, `runs`) and template validators such as `failsOnStub` (`DevTeam`) and `passesDryRun` (`incident`). All of them go through one function, `run_python` in `src/agentm2m/auto/sandbox.py`, which supports four modes (Table 16).

**Table 16.** Sandbox modes (`AGENTM2M_SANDBOX`).

Mode	What it does	Isolating?
process (default)	A child process in a fresh temporary directory, with limits on CPU time, memory (2 GB), file size (64 MB) and wall-clock time, an isolated interpreter ( <code>python -I</code> ) and a minimal environment. It can still read what the user can read and use the network.	no
bwrap	bubblewrap: no network, its own process, IPC and user namespaces, a fresh <code>/proc</code> , only system directories and the Python installation mounted read-only, plus the run's temporary directory. The data directory and other tenants are invisible. (If the Python environment lives under your home directory, that part of home is visible, read-only.)	yes
command	Prefix every run with <code>AGENTM2M_SANDBOX_CMD</code> (for example an <code>nsjail</code> or <code>firejail</code> call).	assumed (the wrapper's job)
off	Never execute. A submission whose validator would execute code is refused (a tool error over MCP, 403 over REST) without spending an attempt. An unknown mode also means off.	—

## 13.2 Fail closed on a shared server

On your own machine, process is acceptable: you run your own team’s code with your own rights. On a shared server it is not: one tenant’s code could read another tenant’s files. The service therefore works like this:

1. At start-up it declares that isolation is required; from then on the process mode is refused.
2. The first time execution status is needed, it *probes* the configured sandbox by running a tiny program in it, and remembers the result for the life of the process.
3. If the probe fails or there is no isolating sandbox, code execution is disabled: the health endpoint and the Status tab say why (Listing 42), and submissions that would execute code are refused (Listing 43). Checking, task and team submission and status keep working.
4. Only an operator who trusts every key holder can override this with `AGENTM2M_ALLOW_UNSANDBOXED_EXEC=1`.

This is *failing closed*, Saltzer and Schroeder’s *fail-safe defaults* [9]: when a safety mechanism is unavailable, the dangerous feature switches off, not the safety. The container image installs bubblewrap, sets `AGENTM2M_SANDBOX=bwrap`, and runs the server as an unprivileged user.

**In plain words:** *if the server cannot lock the code in a room, it does not run the code at all.*

**Remark 9 (A refusal is not a verdict).** When the sandbox refuses to run a validator, the engine does *not* count it as a rejected attempt: the refusal is passed on as an error, not turned into a rejection. Otherwise a server without a sandbox would escalate every correct value after  $k$  “failures” that were never judged.

## 13.3 A threat table

**Table 17.** Threats and defences in version 0.3.0 (✓ enforced by code; ○ by documentation or configuration only).

Threat	Defence	
A workspace’s <code>helpers.py</code> is malicious.	Only open trusted workspaces.	○
Paths in rules or uses escape <code>.agentm2m/</code> .	Paths are checked.	✓
An LLM writes Python through <code>team.evolve</code> .	<code>rule_text</code> only creates <code>.agentm2m</code> files.	✓
A rule expression reaches Python internals through double-underscore names such as <code>__class__</code> .	Expressions cannot reach private or double-underscore attributes.	✓
LLM-written code harms the machine.	One sandbox function; resource limits; <code>bwrap</code> on the server.	✓
One tenant reads another’s projects.	Fail-closed isolation; per-tenant directories; project-name check.	✓
A client claims another tenant’s identity.	The tenant header is stripped and set only by the router.	✓
The key database leaks.	Only SHA-256 digests are stored; keys are shown once.	✓
A web page in the user’s browser makes requests to a local service under a forged host name (DNS rebinding [18]).	Opt-in host checking with <code>AGENTM2M_ALLOWED_HOSTS</code> ; otherwise the endpoint relies on API keys.	○
Source data leaks to an LLM provider.	Host mode: nothing leaves Claude Code’s session. Engine LLMs receive only binding prompts (instruction plus footprint).	✓

**Q14. Check your understanding.** Why is the process sandbox good enough when you run the plugin on your own laptop, but refused on the hosted service? What exactly is the difference in whose code runs?

#### Key takeaways.

- Opening a workspace runs its `helpers.py`; treat it like any code in the repository.
- LLM-written code always runs through one sandbox function; the shared server refuses to run it without real isolation (fail closed), and a refusal never counts as a rejection.
- Authority (the tenant) is set only by trusted code; secrets (keys) are stored only as digests.

## 14 Building, testing and releasing the plugin

### 14.1 One version, six places

The version number `0.3.0` appears in six files: the engine's `pyproject.toml` and `src/agentm2m/__init__.py` (`__version__`), the plugin manifest, the development marketplace entry, the uvx pin in `.mcp.json`, and the pin in the validation hook (and in a seventh, the public marketplace, once it exists). They must agree: the skills and subagents name tools that a specific engine version provides, and the hook must check files with the same engine that runs them. The script `plugin/scripts/bump_version.py` changes all of them at once (`-check` prints “ok: 0.3.0 in 6 places”), and a test fails if they differ.

### 14.2 Scripts

- `validate.sh` runs everything continuous integration (CI) checks: `claude plugin validate -strict` on the plugin and the marketplace, JSON syntax, version consistency, that every template creates a valid workspace, and both test suites.
- `build.sh` builds the engine's *wheel* (an installable archive) and *source distribution*, checks them with `twine` (the Python upload tool), verifies that the wheel contains what the plugin needs at run time (the grammar file, the templates, the `agentm2m-mcp` entry point), installs it in a fresh uvx environment as a smoke test, and zips the plugin.
- `dev-install.sh` validates the plugin, checks that the local engine starts through uvx, registers the development marketplace and installs the plugin from the checkout.
- `release.sh` defaults to a *dry run*: it builds everything and uploads nothing. Publishing (first to TestPyPI, a test copy of the Python Package Index, then to PyPI, then a git tag and a public marketplace entry) must be switched on explicitly with `AGENTM2M_ALLOW_PUBLIC=1`.<sup>1</sup> Listing in Anthropic's plugin directory is a manual form; the script only prepares the entry.

`pyproject.toml` also contains two settings for `uv`. `exclude-newer = "7 days"` makes `uv` refuse dependency versions published in the last week when developers lock or sync *this repository*: a cooldown against freshly compromised packages, a supply-chain defence. It does not affect what `uvx -from agentm2m==...` resolves on a user's machine (we checked: `uv` does not apply a package's own resolver settings to `uvx`); users who want the same cooldown set it in their own `uv` configuration. `cache-keys` makes `uvx` rebuild a local checkout when its source files change, so `AGENTM2M_ENGINE=<checkout>` always runs your latest code.

<sup>1</sup>As of version 0.3.0 the reason is that the paper describing the tool is under double-anonymous review, where reviewers must not learn who the authors are; a public package under the authors' names would reveal them.

### 14.3 Tests

The plugin has its own test suite (plugin/tests/) next to the engine's (tests/). Its five files test different layers:

- `test_plugin_package.py`: the plugin as installed. Manifests are valid and versions agree; every skill has frontmatter and names only real tools; every subagent allowlists only tools the server exposes (the test reads the tool names from `mcp_server.py`); hook scripts are executable and behave on real input, including the silent exit without `uvx`.
- `test_host_mode.py`: the binding protocol. The first run builds structure without values; a prompt contains only the footprint; a rejection gives a reason and a retry prompt; escalation after  $k$  rejections, cleared by a footprint change; a value for an outdated footprint is refused; blocked values unblock.
- `test_change_and_persistence.py`: state survives a reload; impact is exactly the affected values; an identical re-derivation stops propagation, a changed one propagates one hop further; a workflow spread over several processes; instances see each other's writes.
- `test_edit_trace_hot.py`: write rights, engine ownership, atomic edits, trace queries, and team evolution with retroactive obligations.
- `test_mcp_server.py`: the real server over stdio, as Claude Code uses it: the full host workflow, an engine backend, the AutoM2M workflow, and forwarding to a hosted service (including a bad key).

When we ran both suites for this document, the result was *117 passed, 3 skipped*, in about 23 seconds. Testing an MCP server “over stdio” means starting it as a child process and speaking JSON-RPC to it, exactly as we did for Listings 5, 6, 8 and 9.

**Q15. Check your understanding.** Suppose someone bumps the engine to 0.4.0 in `.mcp.json` but forgets the hook. What can go wrong for a user, and which test would catch it before release?

#### Key takeaways.

- The plugin and the engine are released together; one script keeps six version numbers in step and a test enforces it.
- Tests target each layer: the installed package, the binding protocol, persistence, edits, and the real server over stdio.
- The release script is safe by default (dry run).

## 15 Requirements revisited, and what is still open

Section 4 set seven requirements. Table 18 checks each against what we have seen.

Beyond the requirements, these limits matter:

- **Host-mode attribution only locates.** Classifying a fault (Table 13) needs replays by an engine LLM.
- **Escalation semantics differ.** For a hand-written team an escalated value is re-offered only when its footprint changes, so a retry may need a source edit; for an AutoM2M team any admitted repair clears all escalations.
- **Validator strength is not checked.** The hand-written templates use form validators on code, and `failsOnStub` accepts an oracle that asserts nothing;  $\varphi$  can hold for wrong code.
- **AutoM2M is narrow.** Python classes and functions only; one tool (exec); acyclic hand-offs; one writer per view.

**Table 18.** The requirements of Section 4, revisited.

Req.	Status	Evidence	What remains
R1	met	24 tools, 9 slash commands; Sections 10 and 11; <code>test_mcp_server.py</code>	–
R2	met	host mode uses the engine’s own acceptance path (Section 8); <code>test_host_mode.py</code>	no control over which model writes values
R3	partly	workers can call only two tools (Figure 7)	the dispatch message and values the main agent fills itself rest on instructions
R4	partly	state survives reloads and processes; <code>test_change_and_persistence.py</code>	overlapping operations of two processes can lose a reported acceptance (Section 7); AUTO2M files have no lock
R5	partly	the hook re-checks edited team files (Listing 18)	edits through Bash are not seen; the hook is silent without <code>uvx</code> or <code>python3</code>
R6	partly	<code>agentm2m serve</code> ; direct <code>claude mcp add</code> needs nothing installed	the plugin route still runs a local proxy; REST covers AUTO2M only; local-file skills and hooks do not reach the service
R7	partly	fail-closed server, one sandbox function, path and header checks (Table 17)	locally, process mode does not isolate; <code>helpers.py</code> is trusted

- **The service is a research preview.** No rate limits, tiers not enforced, in-memory metrics, no verified e-mail addresses, no login.
- **Not yet published.** Until the engine is on PyPI, `AGENTM2M_ENGINE` must point at a checkout.
- **Scale is unmeasured.** State is one JSON file per workspace and every run re-evaluates all matches; we have not measured large teams in the plugin.

## 15.1 Conclusion

A research engine became a tool by keeping one idea at every layer: *an LLM writes, a program decides*. MCP gives Claude Code a precise, typed way to call the engine; host mode lets Claude Code do all the LLM work while the engine keeps structure, acceptance and completion; subagent allowlists and `footprint_version` protect the one property host mode puts at risk, that every value is written from its footprint; hooks catch broken designs the moment they are written; and the hosted service offers the same tools to many users without letting their data or code meet. Where the design falls short, the shortfalls are known, named, and mostly measurable, which is itself what a checked composition is meant to give.

## 16 Exercises

Difficulty: ★ check your reading, ★★ apply, ★★★ design. Hands-on exercises are marked (H); Appendix B shows how to run the engine. Hints are in Appendix F.

1. ★ **(Protocol)** Write the JSON-RPC request that calls `impact` for the change of Section 10, Step 6. Then write the response for the case where `as_agent` names an agent that does not exist. Is it a protocol error or a tool error?
2. ★ **(States)** For each of these situations, give the state of the value in Figure 6 and the status `submit_binding` would return: (a) a value submitted with an outdated `footprint_version`; (b) the third wrong answer on one footprint; (c) a correct answer for a value that is already fresh.
3. ★★ **(Schema)** `model_edit` declares `ops` as `list[dict[str, Any]]`; its docstring describes three kinds of operation. Sketch the JSON Schema the SDK generates. What would you gain and lose with a precise schema for each kind of operation?

4. \*\* (Plugin anatomy) Write a new skill `/agentm2m:explain` that, given an element key, explains in plain words what was derived from it and why. Give its frontmatter (name, description, argument-hint) and the tools it may call.
5. \*\* (Least privilege) Exercise the colleague’s proposal of Q8 further: design a way to give the worker the project’s style guide without breaking change impact.
6. \*\* (Concurrency) Two Claude Code sessions run `/agentm2m:run` on the same project at the same time. List every mechanism of Sections 7 and 8 that keeps the state consistent, and describe one interleaving that ends with a stale status and one that loses a reported acceptance.
7. \*\* (H) (Templates) Initialise the incident template with `AGENTM2M_LLM=mock` and run it. Explain every escalation from the template’s comments and validators.
8. \*\* (H) (Validators) Add a validator to `helpers.py` that rejects an oracle without an assert statement, with a Rejected reason. Use it in `Req2Test` and observe the `retry_prompt` for a bad oracle.
9. \*\*\* (Design) Modify `Operation2CodeEdit` of `DEVTEAM` so that a code body is accepted only if it passes the oracles of its story’s criteria. What must the footprint or the validator’s inputs contain, and which `AUTO2M2M` condition does the modified team now satisfy?
10. \*\* (H) (Service) Start `agentm2m serve` locally, create a key with `curl`, and drive the Wallet task through the REST API up to the first 403. Then set the sandbox so that execution is enabled, explain why your choice is or is not safe for a shared server, and finish the task.
11. \*\* (Security) Show a project name that the rule `[A-Za-z0-9_.-]{1,64}` without `.` and `..` blocks and that could otherwise escape the tenant’s directory. Is `..x` dangerous?
12. \*\*\* (Repair) In Section 11 the delta widened a footprint. Propose a delta that would help a Developer who really lacked information, and say whether it is applied in place, hot, or by rebuilding.
13. \*\*\* (Design critique) The REST API covers only the `AUTO2M2M` tools. Write a one-page design note: should the hand-written-team tools get REST endpoints too? Consider testability, security (`helpers.py`) and the users of each interface.

## A Reference: the 24 tools

---

Arguments in square brackets are optional. “Changes state” means the tool may write to the workspace.

**Table 19.** AGENTM2M tools (hand-written teams, `.agentm2m/`).

Tool	Arguments	What it does	Changes state
<code>team_init</code>	<code>[template]</code> , <code>[force]</code>	Copies a template ( <code>devteam</code> , <code>research</code> , <code>incident</code> ) into <code>.agentm2m/</code> ; <code>force</code> replaces an existing workspace.	yes
<code>team_status</code>	–	Agents and views, hand-offs, trace links, binding states, open items, $\varphi$ .	no
<code>team_validate</code>	–	Parses the spec and every rule module; checks classes, root slots, helper paths; counts matches.	no
<code>model_show</code>	<code>view</code> , <code>[key]</code> , <code>[depth]</code>	Shows a view or one element; marks engine-owned elements.	no
<code>model_edit</code>	<code>view</code> , <code>ops</code> , <code>as_agent</code>	Atomic create/set/delete as the owning agent; rebuilds structure at once.	yes
<code>impact</code>	<code>[view]</code> , <code>ops</code> , <code>as_agent</code>	Previews the obligations of an edit on a scratch copy. No LLM call.	no
<code>run</code>	<code>[max_passes]</code>	Runs every hand-off to a fixpoint; reports created, pending, blocked, escalated, $\varphi$ .	yes
<code>next_bindings</code>	<code>[agent]</code> , <code>[limit]</code>	The next values to fill, each with its prompt and <code>footprint_version</code> . Host mode only; runs the structural phase first.	yes
<code>submit_binding</code>	<code>target_key</code> , <code>binding</code> , <code>value</code> , <code>footprint_version</code>	Judges one value with its validator (Table 10).	yes
<code>trace_query</code>	<code>key</code> , <code>[transitive]</code>	Upstream sources of an element and everything derived from it.	no
<code>team_evolve</code>	<code>agent</code> , <code>view</code> , <code>handoff</code> , <code>rule</code> , <code>view_spec</code> or <code>view_spec_file</code> , <code>[rule_text]</code>	Adds an agent, its view and a hand-off while the team runs. The view is given inline or as a YAML file in <code>.agentm2m/</code> ; <code>rule_text</code> creates the rule file.	yes
<code>acceptance</code>	–	$\varphi$ and the list of what is still open.	no

**Table 20.** AutoM2M tools (builder-proposed teams for a Python task, `.agentm2m/auto/`).

Tool	Arguments	What it does	Changes state
<code>auto_task_set</code>	<code>source</code> , <code>[description]</code>	Lifts a Python skeleton into the Goal view; discards the run state.	yes
<code>auto_check</code>	<code>[team_json]</code> , <code>[naive]</code>	W1–W6 on any typed team, without changing anything. <code>naive</code> uses a weaker, class-level W4 (for comparison only).	no
<code>auto_propose</code>	<code>[mode]</code>	The builder prompt: <code>propose</code> , <code>revise</code> , <code>delta</code> , or <code>auto</code> (picks one).	no*
<code>auto_submit_team</code>	<code>team_json</code>	Checks a typed team; admits it, or applies it as a checked delta (Table 14).	yes
<code>auto_run</code>	–	Runs to a fixpoint and evaluates $\varphi$ .	yes
<code>auto_next_bindings</code>	<code>[agent]</code> , <code>[limit]</code>	The next values to fill (role, task, format, footprint).	yes
<code>auto_submit_binding</code>	<code>target_key</code> , <code>binding</code> , <code>value</code> , <code>footprint_version</code>	The library validators decide; behaviour validators execute the code.	yes
<code>auto_status</code>	–	Task, team, last diagnostics, $\varphi$ and failing clauses, pending values, history.	no
<code>auto_attribute</code>	<code>[limit]</code>	Locates every failed clause; with an engine LLM also classifies it (Table 13).	yes
<code>auto_deliverable</code>	–	The assembled Python code and which methods are accepted.	no
<code>auto_solve</code>	<code>[source]</code>	The whole loop unattended (at most 3 admission rounds, 2 repairs); needs an engine LLM.	yes
<code>auto_reset</code>	–	Deletes <code>.agentm2m/auto/</code> .	yes

\* In mode `delta` without stored fault reports, it first runs attribution.

## B Try it yourself

**With Claude Code.** You need Claude Code, uv (which provides uvx), Python 3.11 or newer, and a checkout of the repository.

**Listing 47.** Code excerpt: installing the plugin from a checkout (adapted from plugin/scripts/dev-install.sh).

```
git clone <repository> agentm2m-repo && cd agentm2m-repo
python -m venv .venv && . .venv/bin/activate && pip install -e ".[dev]"
plugin/scripts/dev-install.sh # validates and installs agentm2m@agentm2m-dev
export AGENTM2M_ENGINE=$PWD # the engine is not on PyPI yet
mkdir /tmp/play && cd /tmp/play && git init
claude # a scratch project, NOT the engine repository
```

In Claude Code, /plugin should list agentm2m, and /mcp should show the agentm2m server as connected. Then type /agentm2m:init devteam and /agentm2m:run, and follow Section 10. If the server does not start, check that uvx -version works and that AGENTM2M\_ENGINE is set in the shell that started Claude Code.

**Without Claude Code.** Everything except the LLM is available from the command line and from Python. The scripts in docs/plugin-walkthroughs/ reproduce every observed output of this document; AGENTM2M\_LLM=mock lets the engine fill values with canned text.

**Listing 48.** Code excerpt: the engine without Claude Code.

```
python docs/plugin-walkthroughs/walkthrough1_devteam.py # Section 10
python docs/plugin-walkthroughs/walkthrough2_autom2m.py # Section 11
python docs/plugin-walkthroughs/mcp_stdio_demo.py /tmp/p # Section 5 (empty dir /tmp/p)
python docs/plugin-walkthroughs/rest_demo.py # Section 12
agentm2m workspace --dir /tmp/q --llm mock init incident && agentm2m workspace --dir /tmp/q --llm mock run
agentm2m auto check teams/devteam_proposal.json
agentm2m auto --dir /tmp/w task docs/plugin-walkthroughs/wallet.py && agentm2m auto --dir /tmp/w propose --
prompt-only
```

**Table 21.** Environment variables.

Variable	Meaning
AGENTM2M_LLM	Who fills values: host (default), anthropic, openai, ollama, mock.
AGENTM2M_MODEL	Model for engine LLMs.
AGENTM2M_MAX_RESAMPLES	$k$ for hand-written teams in every mode (default 3); AutoM2M uses a fixed $k = 3$ .
AGENTM2M_ENGINE	Engine for uvx (default agentm2m==0.3.0); a path for a checkout.
AGENTM2M_PROJECT_DIR	Project the local server opens (default: CLAUDE_PROJECT_DIR, then the current directory).
AGENTM2M_URL, AGENTM2M_API_KEY, AGENTM2M_PROJECT	Forward every tool call to a hosted service.
AGENTM2M_SANDBOX, AGENTM2M_SANDBOX_CMD	process, bwrap, command (with its wrapper), off.
AGENTM2M_DATA_DIR	Server: keys database and tenant projects (default ~/.agentm2m-service).
AGENTM2M_SOLVE_LLM	Server: engine LLM for auto_solve.
AGENTM2M_ALLOW_UNSANDBOXED_EXEC	Server: 1 allows execution without isolation (trusted users only).
AGENTM2M_ALLOWED_HOSTS	Server: opt-in DNS-rebinding protection for /mcp.

## C The admitted team of Walkthrough 2

**Listing 49.** Code excerpt: the admitted typed team (the builder prompt’s worked example, renamed; the fixed Goal view elided).

```
{
 "name": "wallet_team",
 "goal_view": "Goal",
 "agents": [
 {
 "name": "Designer",
 "tools": [],
 "role": "You decide, for one method, the algorithm, edge cases and the fields it reads or updates.",
 },
 {
 "name": "Tester",
 "tools": ["exec"],
 "role": "You turn a method's documentation into executable unit tests with concrete expected values.",
 },
 {
 "name": "Developer",
 "tools": ["exec"],
 "role": "You implement one method following its design so that its tests and examples pass."
 }
],
 "views": {
 "Goal": "<fixed: Task, Method>",
 "Design": {
 "classes": {
 "MethodDesign": {
 "attributes": {
 "name": "string",
 "plan": "string"
 },
 "references": {
 "method": {
 "type": "Goal.Method",
 "required": true
 }
 }
 },
 "MethodTest": {
 "attributes": {
 "name": "string",
 "code": "string"
 },
 "references": {
 "method": {
 "type": "Goal.Method",
 "required": true
 }
 }
 },
 "MethodImpl": {
 "attributes": {
 "name": "string",
 "code": "string"
 },
 "references": {
 "method": {
 "type": "Goal.Method",
 "required": true
 }
 }
 }
 }
 },
 "writes": {
 "Designer": ["Design"],
 "Tester": ["Test"],
 "Developer": ["Code"]
 },
 "handoffs": [
 {
 "name": "Goal2Design",
 "sources": ["Goal"],
 "target": "Design",
 "rules": [
 {
 "name": "Method2Design",
 "from": [
 {
 "var": "m",
 "type": "Goal!Method"
 }
],
 "guard": null,
 "to": [
 {
 "var": "d",
 "type": "Design!MethodDesign"
 }
],
 "bind": {
 "name": "m.name",
 "method": "m"
 },
 "llm": [
 {
 "feature": "plan",
 "prompt": "Describe in a few bullet points how to implement this method: ...",
 "footprint": [
 "m.task.skeleton",
 "m.signature",
 "m.docstring"
],
 "validator": [
 {
 "id": "nonempty"
 }
]
 }
]
 }
]
 },
 {
 "name": "Goal2Test",
 "sources": ["Goal"],
 "target": "Test",
 "rules": [
 {
 "name": "Method2Test",
 "from": [
 {
 "var": "m",
 "type": "Goal!Method"
 }
],
 "guard": null,
 "to": [
 {
 "var": "t",
 "type": "Test!MethodTest"
 }
],
 "bind": {
 "name": "m.name",
 "method": "m"
 },
 "llm": [
 {
 "feature": "code",
 "prompt": "Write unit tests for this method.",
 "footprint": [
 "m.task.skeleton",
 "m.name",
 "m.docstring",
 "m.examples"
],
 "validator": [
 {
 "id": "test_valid",
 "args": {
 "method": "m"
 }
 }
]
 }
]
 }
]
 },
 {
 "name": "Build2Code",
 "... see the rule in Section 11 ..."
 }
]
 },
 "goal": [
 {
 "class": "Method",
 "scope": "all",
 "kind": "checked"
 },
 {
 "class": "Method",
 "scope": "all",
 "kind": "delivered"
 }
],
 "deliverable": {
 "view": "Code",
 "class": "MethodImpl",
 "feature": "code",
 "for": "method"
 },
 "done": [
 "cover(G)",
 "valid",
 "fresh",
 "noObl"
]
}
```

Read it top to bottom: three agents (only those that own behaviour-checked values need exec, W6); one view per agent and one writer per view (W2); three hand-offs from the Goal view; two goal obligations, one saying every method must reach a behaviour check (checked) and one saying every method must be delivered (delivered); the deliverable, which tells the engine where the code of each method is; and the four engine clauses of  $\varphi$  (W5).

## D Glossary

### Protocols and the web.

#### API key

A secret that identifies a caller of a web service; stored by the server only as a digest.

#### ASGI

Python’s standard interface between web servers and web applications.

#### Digest / hash

A short fingerprint of data; equal data gives an equal digest, any change a different one.

#### JSON-RPC

Remote procedure calls in JSON: requests (with id), responses, notifications.

**MCP**

Model Context Protocol: how AI hosts discover and call the tools of servers.

**MCP host, client, server**

The application that owns the LLM; its connection to one server; the program offering tools.

**Primitive**

One of MCP's kinds of capability: tools, resources, prompts.

**REST**

An API style: resources as paths, operations as HTTP methods.

**Server-sent events**

A way for one HTTP response to deliver several messages one after another.

**Session id**

The `Mcp-Session-Id` a Streamable HTTP server may assign; the client repeats it on every request.

**stdio / Streamable HTTP**

MCP's transports: pipes to a child process; HTTP POSTs to one endpoint.

**Tool error vs. protocol error**

A result with `isError: true` that the LLM can act on, vs. a JSON-RPC error (a bug).

**Claude Code and the plugin.****Allowlist**

The tools a subagent may call; all others are refused.

**Context**

Everything the LLM can see in one call.

**Extension point**

A place where a plugin may add behaviour: MCP tools, skills, subagents, hooks.

**Frontmatter**

The YAML header of a skill or subagent file.

**Hook**

A command Claude Code runs at an event (session start, after a tool use); exit code 2 reports a problem.

**Inversion of control**

The host calls the plugin, not the other way round.

**Marketplace**

A list of plugins from which Claude Code installs them.

**Skill**

A plain-language procedure for Claude Code, invoked as a slash command or loaded automatically.

**Subagent**

A helper agent with its own instructions, fresh context and tool allowlist.

**uvx**

Runs a Python package's program in a cached, isolated environment.

**The engine (AGENTM2M).****Binding (structural / stochastic)**

A clause in a rule that fills a field: by a formula, or by an LLM from a footprint.

**Blocked**

A pending value whose footprint contains an upstream value that was never accepted.

**Deferred backend**

The host backend: the engine records pending values instead of sampling.

**Engine-owned**

An element created by a rule; agents may not edit it.

**Escalation**

Giving up on a value after  $k$  rejections on the same footprint.

**Fixpoint**

The state in which another pass of all hand-offs changes nothing.

**Footprint**

The exact data an LLM may see when writing one value.

**footprint\_version**

The digest of a footprint, sent with each prompt and required with each submission.

**Guard**

The condition in parentheses after a rule's source pattern.

**Host mode**

Claude Code, not the engine, writes the values (AGENTM2M\_LLM=host).

**HOT**

Higher-order transformation: a change of the team (new agent, view, hand-off) while it runs.

**Lift binding**

`self <- @llm(...)`: one JSON answer fills several fields.

**Metamodel / model / view**

A form (class diagram); a filled-in form; the model an agent owns.

**Obligation**

A value that must be redone because its footprint changed.

**Pending**

A value ready to be written: never accepted (missing) or stale.

$\varphi$  The acceptance predicate: the engine's definition of "done".

**PendingBinding**

The record of one pending value: place, prompt, attempts, footprint\_version.

**Rule**

"For every source object matching this pattern and guard, create one target object and fill its fields."

**Stamp**

The digest of the footprint a value was accepted for.

**Stale**

A value whose stamp no longer matches its current footprint.

**Target key**

The name of a value's place: `Rule::variable::match`.

**Trace link**

A record that a target object was created from a source object by a rule.

**Validator (form / behaviour)**

A check a value must pass: "does it parse?" vs. "does it pass the tests?".

**Write rights ( $\omega$ )**

Which agent may write which view.

**AUTO M2M.****Builder**

The LLM that writes a typed team; in the plugin, Claude Code (or the team-builder subagent).

**Checker, W1–W6**

The program that admits a typed team only if the six conditions of Table 2 hold.

**cover(G), valid, fresh, noobl**

The four clauses of  $\varphi$  (Section 2.8).

**Delta**

A revised typed team that repairs a running one; applied in place, hot, or by rebuilding.

**Deliverable**

Where the result of each goal (the code of each method) is stored.

**Diagnostic**

One checker message: condition, element, what is wrong.

**Doctest example**

A `>>` line in a docstring with its expected output.

**Fault class**

Sampling, footprint, upstream, specification, validator or coverage (Table 13).

**Goal view / lift**

The fixed view (Task, Method) into which the task's source is turned.

**Stub**

A method with a docstring and a body of `pass, ...` or `raise NotImplementedError`.

**Typed team**

A team design in a fixed JSON format that the checker can read.

**Security and engineering.****Atomic write**

Write a temporary file, then rename it over the old one; readers never see half a file.

**DNS rebinding**

An attack in which a web page reaches a local service through a forged host name.

**Dry run**

Do everything except the irreversible step (here: uploading).

**Fail closed**

When a safety mechanism is unavailable, switch off the risky feature, not the safety.

**Least privilege**

Give each component only the rights it needs.

**Lost update**

Two overlapping read-modify-write operations; one write silently undoes the other.

**Namespace (Linux)**

A private view of the network, process list or file system for one process.

**Optimistic concurrency**

Work without locking; check at the end that nothing changed underneath.

**Proxy**

An object with the same interface as another, which forwards calls to it.

**Sandbox**

An environment that restricts what executed code can do.

**Tenant**

One customer (here: one API key) of a shared service, whose data is kept apart.

## E Answers to the questions

**Q1 (JSON-RPC ids).** Responses may arrive in a different order than the requests were sent, for example when a quick call overtakes a slow one. Without the `id` the client could attach the result of one call to the other, for instance treat the result of `impact` as that of `run`.

**Q2 (2 versus 3 matches).** `Story20operation` has the guard `s.status = #accepted`, which excludes the draft `S3`. `Criterion2TestCase` has no guard, so it matches all three criteria, including the draft's `S3.1`.

**Q3 (descriptions).** The LLM would expect a value and might report “the value is accepted” without reading the status, or treat a status object as the value. No program would notice: the schema describes only the arguments, and the description is never checked against the code. That is why descriptions list every outcome and live next to the code as docstrings.

**Q4 (extension points).** (a) A `PostToolUse` hook. (b) A skill. (c) A subagent whose allowlist lacks `Read`, `Grep` and `Bash`. (d) An MCP tool (`impact`), which a skill can call.

**Q5 (lost update).** Before writing, `A`'s save notices that the fingerprint changed and reloads the state from disk, which contains `B`'s change but not `A`'s in-memory acceptance; it then writes that. So `B`'s change survives and `A`'s is lost. It is acceptable because the state stays consistent: `A`'s value is simply missing again and is offered by the next `next_bindings`. What is lost is work, not correctness.

**Q6 (why not call Claude Code from generate).** The engine runs *inside* a tool call that Claude Code made: Claude Code is waiting for the tool to return. The server cannot ask Claude Code's LLM a question in the middle of that call (and MCP tools are not given a way to do so here); and the value should be written by a subagent with a fresh, restricted context, which only Claude Code can start. So the tool returns the prompt, and Claude Code answers in a later call.

**Q7 (outdated submission).** `stale`. The engine recomputes the digest of the current footprint, which now contains the new signature and criterion; it differs from the submitted `footprint_version`. Accepting the code would stamp it as derived from information it never saw, and a later change would not know that it must be redone.

**Q8 (Read for the worker).** A stamp records the digest of the footprint and promises “this value depends on this data and nothing else”. A worker that also reads files could let a style rule, or any other file, influence the value. When that file changes, no stamp changes, so the value is not redone: change impact is no longer exact.

**Q9 (compile-only validator).** `compiles()` on `CodeEdit.body` checks only form. `W4` would reject the design: the Code view reaches no behaviour check.

**Q10 (trace links versus impact).** `Story20operation` matches a `UserStory`, not a `Criterion`, so its trace link names `UserStory#S2`; the criterion enters only through the *footprint* `s.criteria` (Figure 8). Obligations are found by comparing stamps with current footprints, not by following trace links, so the signature and the code body are found anyway. `trace_query` on `UserStory#S2` shows `op_s2` and its code edit.

**Q11 (AUTO2M2 walkthrough).** (a) Their footprints contain `d.plan` and `t.code`, which had never been accepted. (b) The delta changed only the rule `Method2Impl`; the footprints of the plans and tests did not change, so their stamps stayed fresh. The deposit code is written by the changed rule, whose footprint now also contains the examples, so its stamp no longer matched. (c) Any admitted in-place delta, for example one that only rewords the Developer's prompt: every in-place or hot delta clears all escalations.

**Q12 (spoofed tenant header).** A user with *any* valid key, for example a free one of their own, could send `x-agentm2m-tenant: <victim's key id>` and read, change or delete the victim's projects. The attacker needs a valid key and the victim's key id. Key ids are random, but they are identifiers, not secrets: they appear in key records, logs and directory names. Security must not depend on keeping them hidden, and the router makes sure it does not.

**Q13 (forwarding loop).** The server would forward every incoming call to itself, which would forward it again, and so on, until a timeout or resource limit ended the chain. No call would ever execute.

**Q14 (process sandbox).** Locally, the code belongs to your own team on your own machine; it can do nothing you could not do yourself, and the resource limits stop accidents. On a shared server the code comes from many users; a same-user process could read the data directory with every tenant’s projects and the keys database. Only a sandbox that hides those files is acceptable.

**Q15 (version skew).** The MCP server would run 0.4.0 while the hook validates with 0.3.0: a rule or typed team that 0.4.0 accepts might be reported as broken after every edit, or the reverse. The version-consistency test in `test_plugin_package.py` (and `bump_version.py -check` in `validate.sh`) fails before release.

## F Hints for the exercises

---

1. Copy the shape of Listing 9; the unknown agent gives a tool error.
2. (a) The value stays where it was; status stale. (b) escalated. (c) `already_accepted`.
3. A list of objects with a string field `op` and free other fields. A precise schema (one per kind) lets the client reject bad operations early, at the cost of a larger, harder-to-read schema.
4. Allowed tools: `trace_query`, `model_show`, `impact`; read-only, so it can be user-invocable.
5. Put the style guide into the *footprint*, for example as an attribute of a view that the rule reads; then a change of the guide is a footprint change.
6. Use the lost-update table of Section 7 and the `footprint_version` story of Section 8.
7. The mock’s canned text is not a runnable script, so `passesDryRun` rejects every `dryRun` value (the rule’s comment says so).
8. Parse the oracle with Python’s `ast` module and look for an `Assert` node; return `Rejected("...")` otherwise.
9. The validator must see the oracles: add a reference from the operation to its criteria’s test cases, or a helper that collects them, and run them against the body. W4.
10. `AGENTM2M_SANDBOX=bwrap` if `bubblewrap` works on your machine; otherwise `AGENTM2M_ALLOW_UNSANDBOXED_EXEC=1`, safe only if you are the only key holder.
11. `..` and `..:` `tenants/<id>/..` is the tenants directory. `..x` is an ordinary name inside the tenant’s directory.
12. For example, add a Reviewer view with examples of edge cases that `Method2Impl` reads: a new view and hand-off are added, so the delta is hot.
13. Compare with Section 7 (one service layer) and Section 13 (helpers run code).

## References

---

- [1] Anonymous, “When the team writes itself: Composition defects in automatically assembled LLM agent teams, and how model transformations can check them (teaching edition).” Companion document, `docs/autom2m-explained-v1.tex` in the artifact, 2026.
- [2] Anonymous, “AgentM2M: M2M transformations for agentic collaborations in software engineering,” 2026. Under review; anonymised for double-anonymous review. Prototype and pilot data in the anonymous artifact.

- [3] Anthropic, “Claude code documentation: Plugins, skills, subagents, hooks and MCP.” Online documentation, 2025.
- [4] JSON-RPC Working Group, “JSON-RPC 2.0 specification.” Specification, 2010.
- [5] R. T. Fielding, M. Nottingham, and J. Reschke, “HTTP semantics.” RFC 9110, IETF, 2022.
- [6] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [7] F. Jouault, F. Allilaire, J. Bézivin, and I. Kurtev, “ATL: A model transformation tool,” *Science of Computer Programming*, vol. 72, no. 1–2, pp. 31–39, 2008.
- [8] Containers project, “Bubblewrap: Low-level unprivileged sandboxing tool.” Software, 2024.
- [9] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [10] Microsoft, “Language server protocol specification.” Specification, 2022.
- [11] Anthropic, “Model context protocol.” Specification, 2024.
- [12] Model Context Protocol, “Model context protocol specification, version 2025-06-18.” Specification, 2025.
- [13] Astral, “uv: An extremely fast Python package and project manager.” Software documentation, 2025.
- [14] H. T. Kung and J. T. Robinson, “On optimistic methods for concurrency control,” *ACM Transactions on Database Systems*, vol. 6, no. 2, pp. 213–226, 1981.
- [15] M. Tisi, F. Jouault, P. Fraternali, S. Ceri, and J. Bézivin, “On the use of higher-order model transformations,” in *Model Driven Architecture – Foundations and Applications*, vol. 5562 of *Lecture Notes in Computer Science*, pp. 18–33, Springer, 2009.
- [16] Django Software Foundation, “ASGI (asynchronous server gateway interface) specification.” Specification, 2019.
- [17] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [18] C. Jackson, A. Barth, A. Bortz, W. Shao, and D. Boneh, “Protecting browsers from DNS rebinding attacks,” in *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS)*, pp. 421–431, 2007.